

Computer Vision

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

University of Bucharest, 2nd semester, 2025-2026

Course structure

1. Low-level vision: Features and filters

Linear filters, color, texture, edge detection, template matching

2. Mid-level vision: Grouping and fitting

Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. Mid-level vision: Multiple views

Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. High-level vision: Object recognition

Object classification, object detection, part based models, bovw models

5. High-level vision: Video understanding

Object tracking, background subtraction, motion descriptors, optical flow

Comparison of Feature Detectors

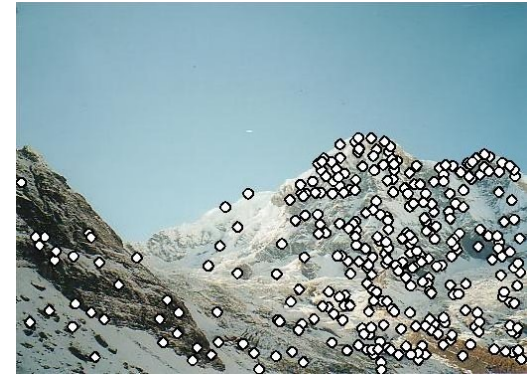
Table 7.1 Overview of feature detectors.

Feature Detector	Corner	Blob	Region	Rotation invariant	Scale invariant	Affine invariant	Repeatability	Localization accuracy	Robustness	Efficiency
Harris	✓			✓			+++	+++	+++	++
Hessian		✓		✓			++	++	++	+
SUSAN	✓			✓			++	++	++	+++
Harris-Laplace	✓	(✓)		✓	✓		+++	+++	++	+
Hessian-Laplace	(✓)	✓		✓	✓		+++	+++	+++	+
DoG	(✓)	✓		✓	✓		++	++	++	++
SURF	(✓)	✓		✓	✓		++	++	++	+++
Harris-Affine	✓	(✓)		✓	✓	✓	+++	+++	++	++
Hessian-Affine	(✓)	✓		✓	✓	✓	+++	+++	+++	++
Salient Regions	(✓)	✓		✓	✓	(✓)	+	+	++	+
Edge-based	✓			✓	✓	✓	+++	+++	+	+
MSER			✓	✓	✓	✓	+++	+++	++	+++
Intensity-based			✓	✓	✓	✓	++	++	++	++
Superpixels			✓	✓	(✓)	(✓)	+	+	+	+

Local features: pipeline

1. Detection: where?

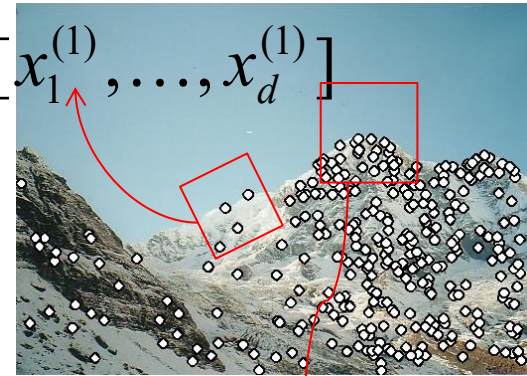
- Identify repeatable and distinctive points in the image
- Typically: corners, blobs, high-texture regions
- Goal: same points should be found across different views
- Examples: Harris, Harris-Laplace, DOG (SIFT), etc.



2. Description: what?

- For each keypoint, extract a feature vector
- Encodes the local appearance around the point
- Should be:
 - Invariant (scale, rotation, illumination)
 - Discriminative
- Examples: SIFT (128D), SURF, ORB, etc.

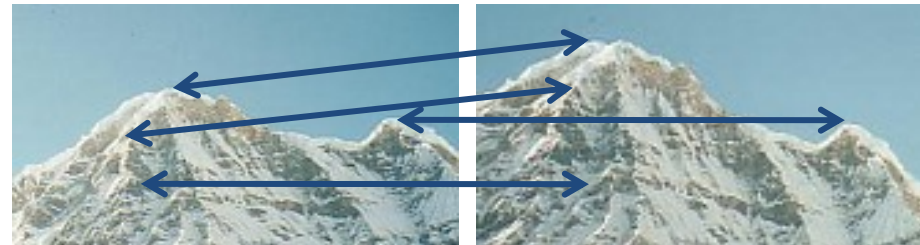
$$\mathbf{x}_1 = [x_1^{(1)}, \dots, x_d^{(1)}]$$



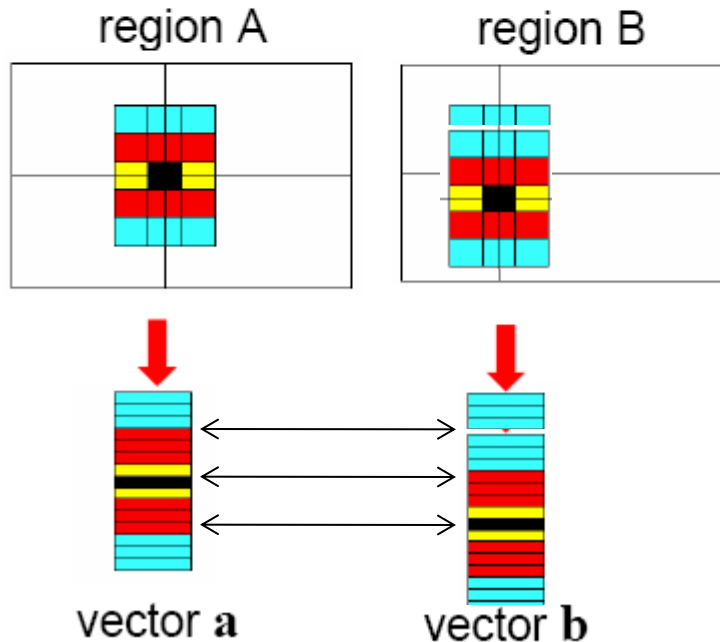
$$\mathbf{x}_2 = [x_1^{(2)}, \dots, x_d^{(2)}]$$

3. Matching: which correspondence to which?

- Compare descriptors across images
- Use a distance metric (e.g., Euclidean distance)
- Often combined with: Ratio test (Lowe), RANSAC (geometric verification)



Raw patches as local descriptors



The simplest way to describe the neighborhood around a keypoint is to write down the list of intensities to form a feature vector.

But this is very sensitive to even small shifts, rotations.

Local Descriptors

- The ideal descriptor should be
 - Robust
 - Distinctive
 - Compact
 - Efficient
- Most available descriptors focus on edge/gradient information
 - Capture texture information
 - Color rarely used



David Lowe

Professor Emeritus, Computer Science Dept., [University of British Columbia](#)
Verified email at cs.ubc.ca - [Homepage](#)

[Computer Vision](#) [Object Recognition](#)



TITLE

CITED BY

YEAR

[Distinctive image features from scale-invariant keypoints](#)

DG Lowe

International journal of computer vision 60 (2), 91-110

81375

2004

[Object recognition from local scale-invariant features](#)

DG Lowe

International Conference on Computer Vision, 1999, 1150-1157

27264

1999

[Fast Approximate Nearest Neighbors with Automatic Algorithm Configuration.](#)

M Muja, DG Lowe

VISAPP (1) 2, 331-340

4496

2009

[Automatic panoramic image stitching using invariant features](#)

M Brown, DG Lowe

International Journal of Computer Vision 74 (1), 59-73

3925

2007

[Unsupervised learning of depth and ego-motion from video](#)

T Zhou, M Brown, N Snavely, DG Lowe

Proceedings of the IEEE Conference on Computer Vision and Pattern ...

3718

2017

[Perceptual Organization and Visual Recognition](#)

DG Lowe

Kluwer Academic Publishers, Boston

2226

*

1985

[Three-dimensional object recognition from single two-dimensional images](#)

DG Lowe

Artificial intelligence 31 (3), 355-395

2119

1987

Distinctive Image Features from Scale-Invariant Keypoints

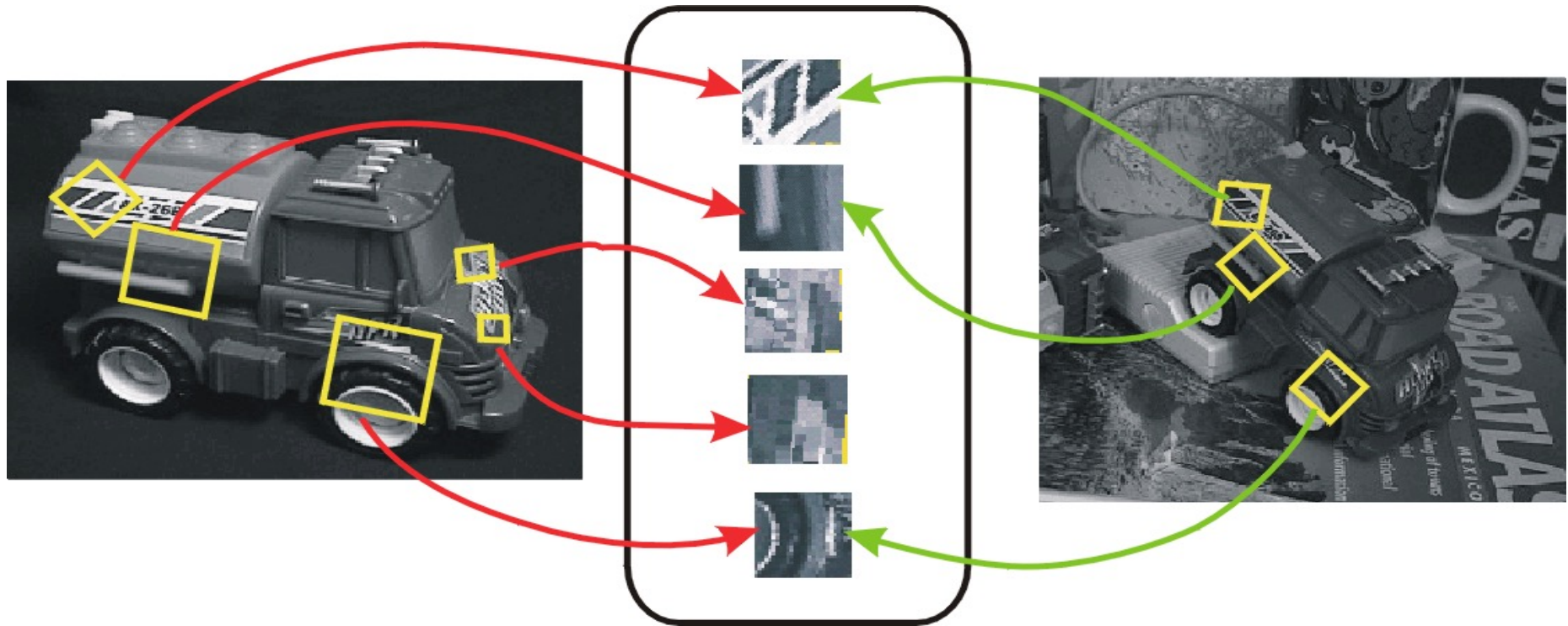
Abstract

This paper presents a method for extracting distinctive invariant features from images that can be used to perform reliable matching between different views of an object or scene. The features are invariant to image scale and rotation, and are shown to provide robust matching across a substantial range of affine distortion, change in 3D viewpoint, addition of noise, and change in illumination. The features are highly distinctive, in the sense that a single feature can be correctly matched with high probability against a large database of features from many images. This paper also describes an approach to using these features for object recognition. The recognition proceeds by matching individual features to a database of features from known objects using a fast nearest-neighbor algorithm, followed by a Hough transform to identify clusters belonging to a single object, and finally performing verification through least-squares solution for consistent pose parameters. This approach to recognition can robustly identify objects among clutter and occlusion while achieving near real-time performance.

SIFT properties

- Invariant to
 - Scale
 - Rotation
- Partially invariant to
 - Illumination changes
 - Camera viewpoint
 - Occlusion, clutter

From keypoint detection to feature description



- Detection is *covariant*:
$$\text{features}(\text{transform}(\text{image})) = \text{transform}(\text{features}(\text{image}))$$
- Description is *invariant*:
$$\text{features}(\text{transform}(\text{image})) = \text{features}(\text{image})$$

Major stages for computing SIFT features

1. **Scale-space extrema detection:** The first stage of computation searches over all scales and image locations. It is implemented efficiently by using a difference-of-Gaussian function to identify potential interest points that are invariant to scale and orientation.
2. **Keypoint localization:** At each candidate location, a detailed model is fit to determine location and scale. Keypoints are selected based on measures of their stability.
3. **Orientation assignment:** One or more orientations are assigned to each keypoint location based on local image gradient directions. All future operations are performed on image data that has been transformed relative to the assigned orientation, scale, and location for each feature, thereby providing invariance to these transformations.
4. **Keypoint descriptor:** The local image gradients are measured at the selected scale in the region around each keypoint. These are transformed into a representation that allows for significant levels of local shape distortion and change in illumination.

This approach has been named the Scale Invariant Feature Transform (SIFT), as it transforms image data into scale-invariant coordinates relative to local features.

Major stages for computing SIFT features

1. **Scale-space extrema detection:** The first stage of computation searches over all scales and image locations. It is implemented efficiently by using a difference-of-Gaussian function to identify potential interest points that are invariant to scale and orientation.
2. **Keypoint localization:** At each candidate location, a detailed model is fit to determine location and scale. Keypoints are selected based on measures of their stability.
3. **Orientation assignment:** One or more orientations are assigned to each keypoint location based on local image gradient directions. All future operations are performed on image data that has been transformed relative to the assigned orientation, scale, and location for each feature, thereby providing invariance to these transformations.
4. **Keypoint descriptor:** The local image gradients are measured at the selected scale in the region around each keypoint. These are transformed into a representation that allows for significant levels of local shape distortion and change in illumination.

This approach has been named the Scale Invariant Feature Transform (SIFT), as it transforms image data into scale-invariant coordinates relative to local features.

Scale space used by SIFT

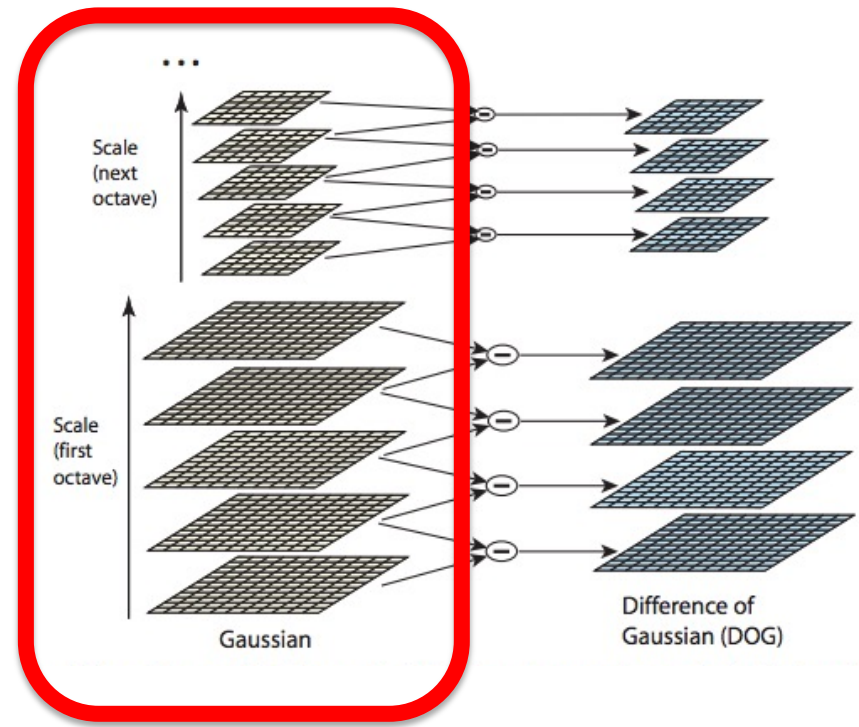
The **scale space** is a *multi-resolution representation* of the image, built by progressively **blurring** and **downsampling** it to detect features that are **invariant to scale**.

The **scale space** used by SIFT is defined as a function:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

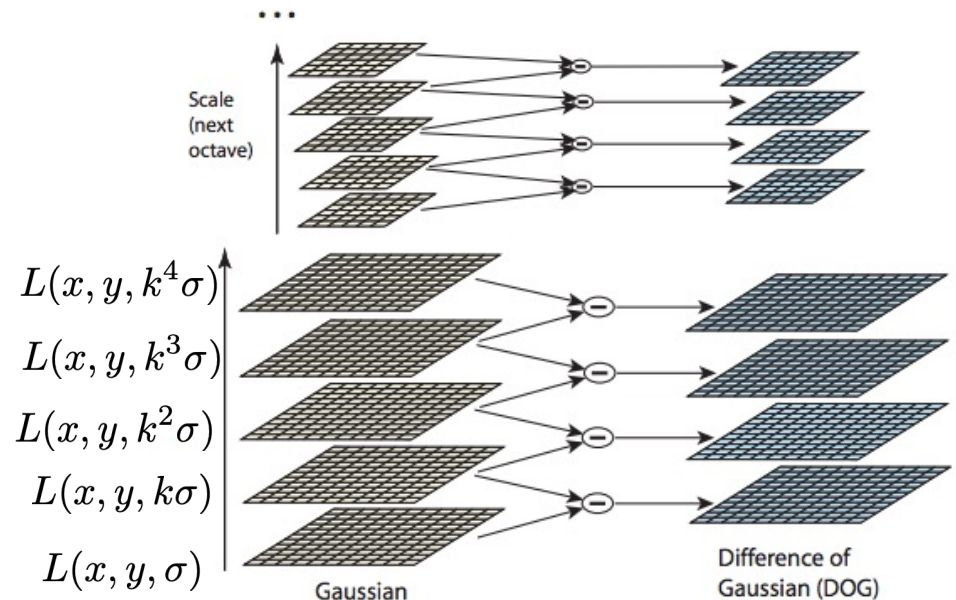
where:

- $I(x, y)$ is the input image
- $G(x, y, \sigma)$ is a 2D **Gaussian kernel** with standard deviation σ
- $L(x, y, \sigma)$ is the **blurred image at scale σ**



Scale space used by SIFT

1. **Start with a base image:** either the original image or a version pre-blurred with $\sigma = 1.6$ (as recommended by Lowe).
2. **Generate multiple blurred versions** by convolving with Gaussians of increasing σ . Blurring simulates how features appear at different zoom levels.



Scale space used by SIFT

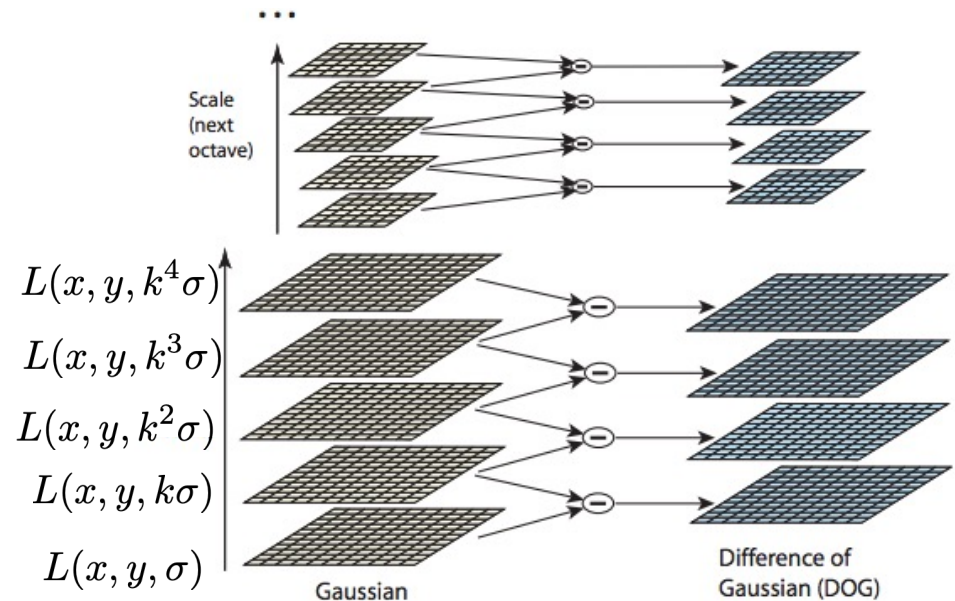
1. **Start with a base image:** either the original image or a version pre-blurred with $\sigma = 1.6$ (as recommended by Lowe).
2. **Generate multiple blurred versions** by convolving with Gaussians of increasing σ . Blurring simulates how features appear at different zoom levels.
3. **Group these into "octaves":** each **octave** spans a **doubling of σ** (e.g., $\sigma = 1.6 \rightarrow 3.2 \rightarrow 6.4 \rightarrow \dots$). At the start of a new octave, the image is **downsampled by 2** in width and height.
4. **Within each octave, compute s+3 blurred images:**
 - common choice: $s = 3$ or $4 \rightarrow 6$ to 7 blurred images per octave.
 - the extra (+3) images are needed because the **Difference of Gaussians** is computed between adjacent scales + local maxima search

Sampling in scale space

The scale levels within an octave are spaced **logarithmically**:

$$\sigma_i = \sigma_0 \cdot k^i, \text{ where } k = 2^{\frac{1}{s}}$$

This ensures even sampling in **$\log(\sigma)$** , which aligns with the fact that humans perceive scale changes logarithmically.



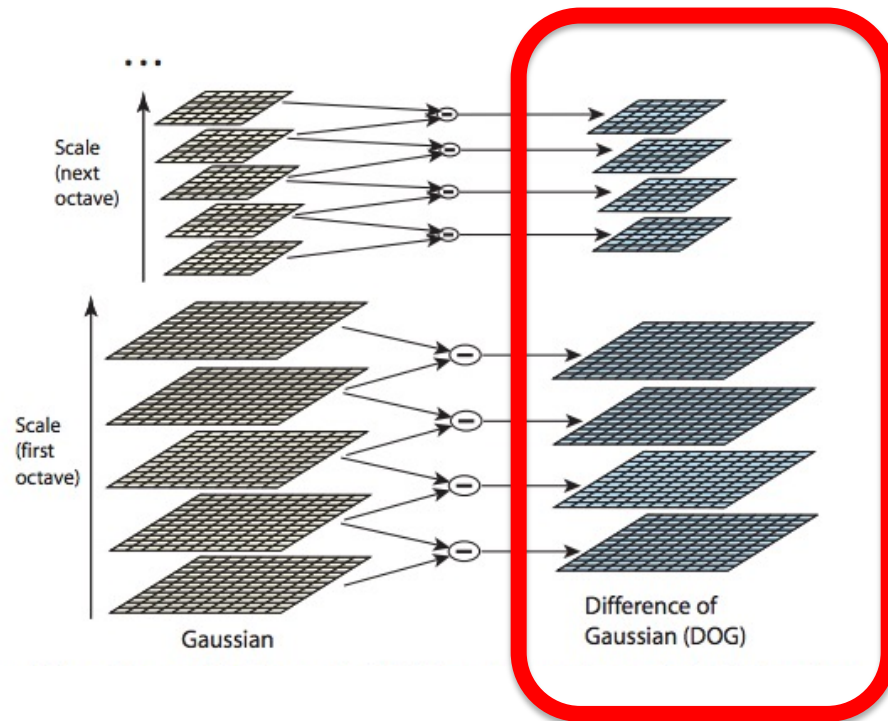
DoG scale space

Instead of directly using $L(x,y,\sigma)$, SIFT computes the **Difference of Gaussians (DoG)**:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

The DoG approximates the **Laplacian of Gaussian (LoG)**, which is computationally more expensive.

The DoG operator highlights **blob-like structures**, and extrema across both scale and space indicate potential keypoints.



1. Scale-space extrema detection

Goal:

Find points that are **local maxima/minima** in scale (σ) and space (x,y).

Method:

1. Construct a Gaussian scale space:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y), \text{ where } G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

2. Approximate the Laplacian of Gaussian (LoG) using Difference of Gaussians (DoG):

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

3. Find 3D extrema: a point (x, y, σ) is a keypoint candidate if it is a local extremum in a $3 \times 3 \times 3$ neighbourhood.

1. Scale-space extrema detection

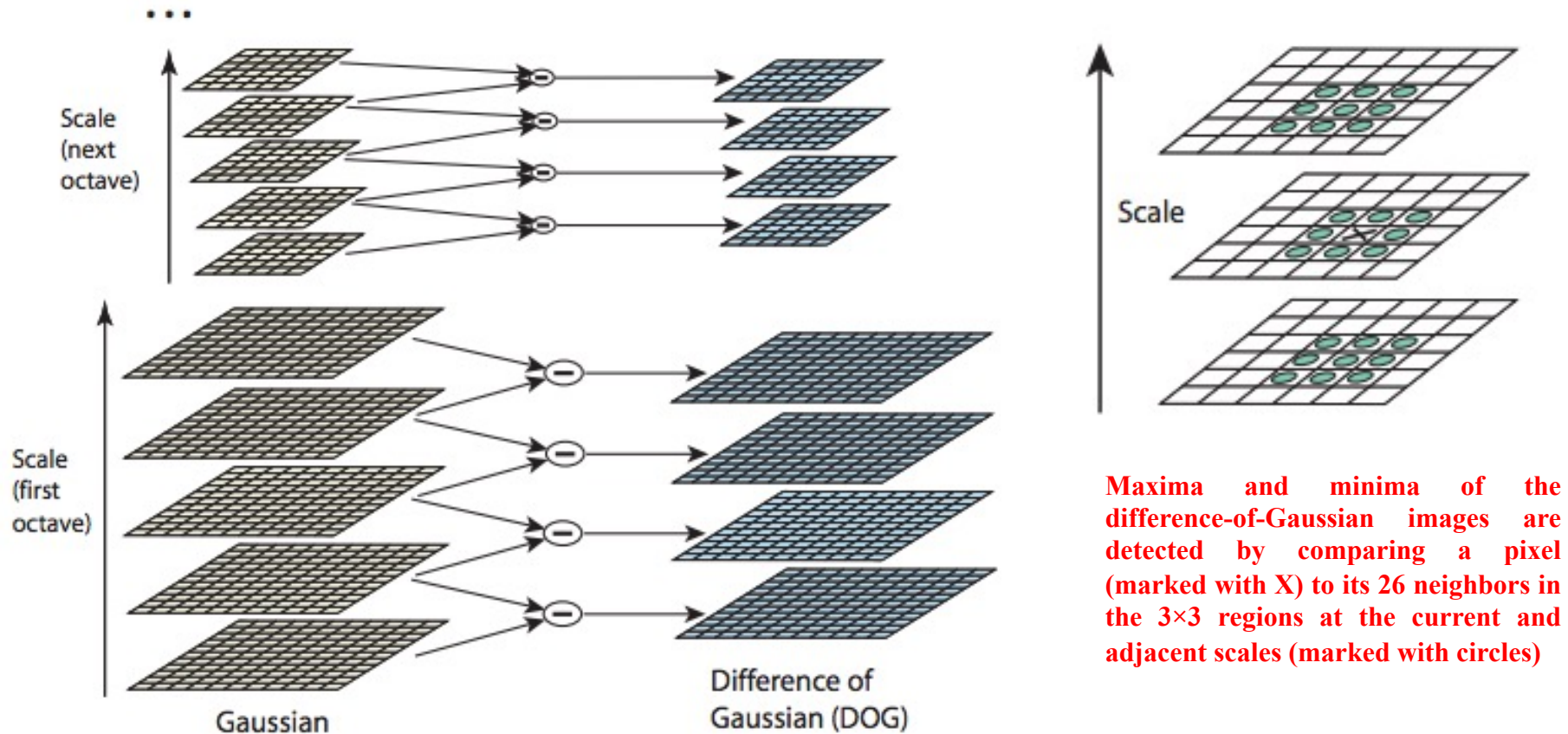


Figure 1: For each octave of scale space, the initial image is repeatedly convolved with Gaussians to produce the set of scale space images shown on the left. Adjacent Gaussian images are subtracted to produce the difference-of-Gaussian images on the right. After each octave, the Gaussian image is down-sampled by a factor of 2, and the process repeated.

$$\begin{aligned}
 D(x, y, \sigma) &= (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y) & G(x, y, k\sigma) - G(x, y, \sigma) &\approx (k - 1)\sigma^2 \nabla^2 G. \\
 &= L(x, y, k\sigma) - L(x, y, \sigma).
 \end{aligned}$$

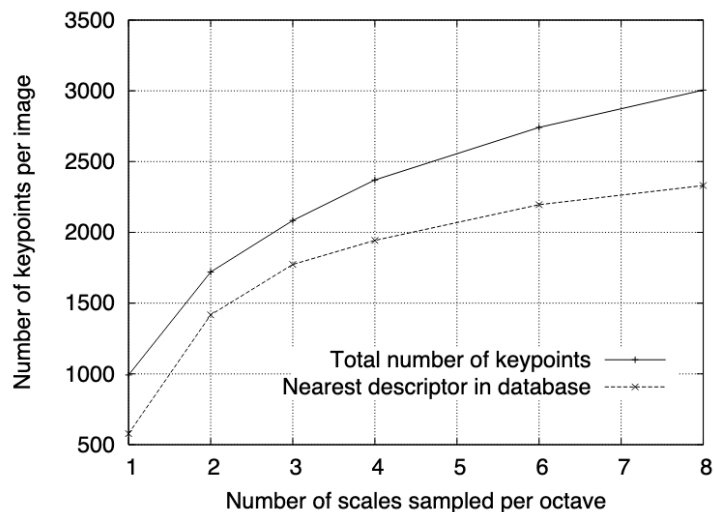
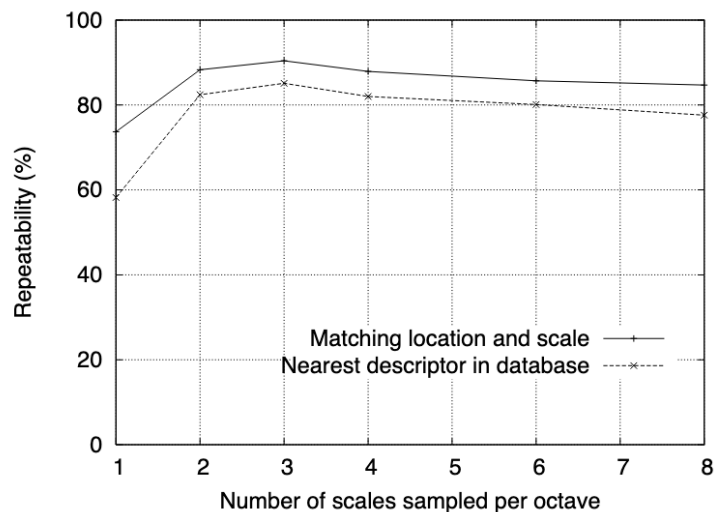


Figure 3: The top line of the first graph shows the percent of keypoints that are repeatably detected at the same location and scale in a transformed image as a function of the number of scales sampled per octave. The lower line shows the percent of keypoints that have their descriptors correctly matched to a large database. The second graph shows the total number of keypoints detected in a typical image as a function of the number of scale samples.

The experimental determination of sampling frequency that maximizes extrema stability is shown in Figures 3 and 4. These figures (and most other simulations in this paper) are based on a matching task using a collection of 32 real images drawn from a diverse range, including outdoor scenes, human faces, aerial photographs, and industrial images (the image domain was found to have almost no influence on any of the results). Each image was then subject to a range of transformations, including rotation, scaling, affine stretch, change in brightness and contrast, and addition of image noise. Because the changes were synthetic, it was possible to precisely predict where each feature in an original image should appear in the transformed image, allowing for measurement of correct repeatability and positional accuracy for each feature.

likely to be useful for recognition and matching tasks). As this graph shows, the highest repeatability is obtained when sampling 3 scales per octave, and this is the number of scale samples used for all other experiments throughout this paper.

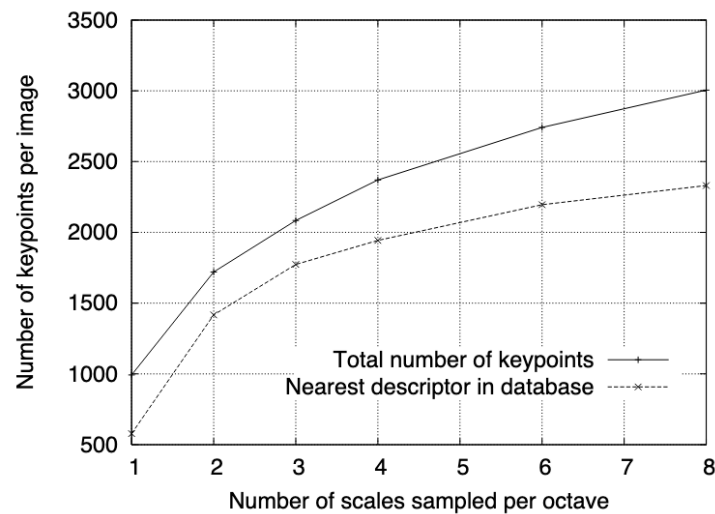
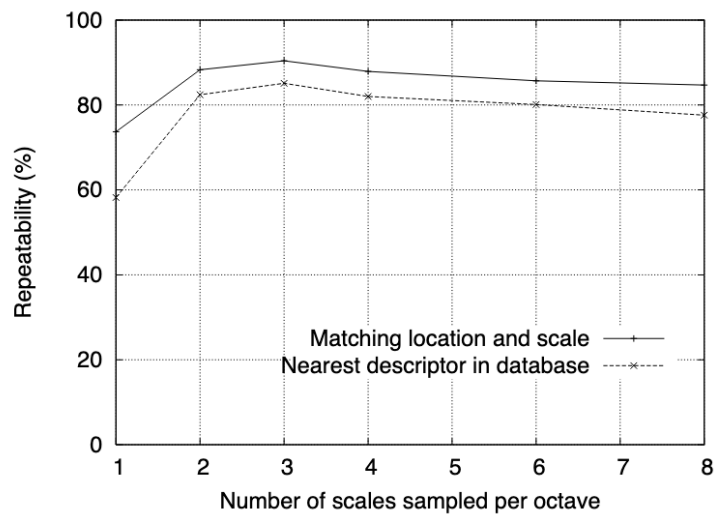


Figure 3: The top line of the first graph shows the percent of keypoints that are repeatably detected at the same location and scale in a transformed image as a function of the number of scales sampled per octave. The lower line shows the percent of keypoints that have their descriptors correctly matched to a large database. The second graph shows the total number of keypoints detected in a typical image as a function of the number of scale samples.

The experimental determination of sampling frequency that maximizes extrema stability is shown in Figures 3 and 4. These figures (and most other simulations in this paper) are based on a matching task using a collection of 32 real images drawn from a diverse range, including outdoor scenes, human faces, aerial photographs, and industrial images (the image domain was found to have almost no influence on any of the results). Each image was then subject to a range of transformations, including rotation, scaling, affine stretch, change in brightness and contrast, and addition of image noise. Because the changes were synthetic, it was possible to precisely predict where each feature in an original image should appear in the transformed image, allowing for measurement of correct repeatability and positional accuracy for each feature.

likely to be useful for recognition and matching tasks). As this graph shows, the highest repeatability is obtained when sampling 3 scales per octave, and this is the number of scale samples used for all other experiments throughout this paper.

s = 3

Major stages for computing SIFT features

1. **Scale-space extrema detection:** The first stage of computation searches over all scales and image locations. It is implemented efficiently by using a difference-of-Gaussian function to identify potential interest points that are invariant to scale and orientation.
2. **Keypoint localization:** At each candidate location, a detailed model is fit to determine location and scale. Keypoints are selected based on measures of their stability.
3. **Orientation assignment:** One or more orientations are assigned to each keypoint location based on local image gradient directions. All future operations are performed on image data that has been transformed relative to the assigned orientation, scale, and location for each feature, thereby providing invariance to these transformations.
4. **Keypoint descriptor:** The local image gradients are measured at the selected scale in the region around each keypoint. These are transformed into a representation that allows for significant levels of local shape distortion and change in illumination.

This approach has been named the Scale Invariant Feature Transform (SIFT), as it transforms image data into scale-invariant coordinates relative to local features.

Keypoint refinement

After detecting candidate keypoints as extrema in the DoG scale space, the algorithm performs a **keypoint refinement step** to eliminate:

1. **Low-contrast points** (i.e., weak responses)
2. **Edge-like points** (unstable localizations, such as along ridges or edges)

Keypoint refinement

After detecting candidate keypoints as extrema in the DoG scale space, the algorithm performs a **keypoint refinement step** to eliminate:

1. **Low-contrast points** (i.e., weak responses)
2. **Edge-like points** (unstable localizations, such as along ridges or edges)

Eliminating **low-contrast** keypoints

- low-contrast points are sensitive to noise and small image variations → unreliable for matching.
- for each candidate extremum (x, y, σ) , SIFT fits a **3D quadratic function** (in variables x, y, σ) to the DoG values in a $3 \times 3 \times 3$ neighborhood to refine the location - compute a new maximum $\hat{x}$:

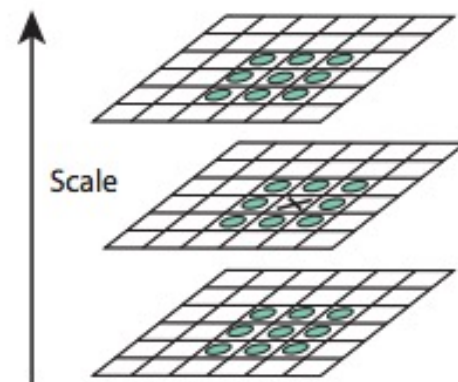
$$D(\mathbf{x}) = D + \frac{\partial D^T}{\partial \mathbf{x}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

quadratic function

$$\hat{\mathbf{x}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{x}^2} \frac{\partial D}{\partial \mathbf{x}}$$

new maximum

- discard the new maximum $\hat{x}$ found if $|D(x)| < 0.03$ (low contrast threshold)



$$\frac{\partial D}{\partial \mathbf{x}} = \left[\frac{\partial D}{\partial x}, \frac{\partial D}{\partial y}, \frac{\partial D}{\partial \sigma} \right]^T$$

$$\frac{\partial^2 D}{\partial \mathbf{x}^2} = \begin{bmatrix} D_{xx} & D_{xy} & D_{x\sigma} \\ D_{xy} & D_{yy} & D_{y\sigma} \\ D_{x\sigma} & D_{y\sigma} & D_{\sigma\sigma} \end{bmatrix}$$

Keypoint refinement

After detecting candidate keypoints as extrema in the DoG scale space, the algorithm performs a **keypoint refinement step** to eliminate:

1. **Low-contrast points** (i.e., weak responses)
2. **Edge-like points** (unstable localizations, such as along ridges or edges)

Eliminating **edge-responses** keypoints

- the DoG function has strong responses along **edges** as well as **blobs**, but edges are unstable for localization and matching.
- around each candidate point, SIFT computes the **2×2 Hessian matrix** (spatial second derivatives) of the DoG at that scale:

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

$$\text{Tr}(\mathbf{H}) = D_{xx} + D_{yy} = \alpha + \beta,$$

$$\text{Det}(\mathbf{H}) = D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta.$$

- consider α the eigenvalue with the largest magnitude and β be the smaller one, $\alpha = r\beta$
- decide if the current keypoint is an edge if $r > 10$ ($\alpha \gg \beta$, like in Harris), computing

$$\frac{\text{Tr}(\mathbf{H})^2}{\text{Det}(\mathbf{H})} = \frac{(\alpha + \beta)^2}{\alpha\beta} = \frac{(r\beta + \beta)^2}{r\beta^2} = \frac{(r + 1)^2}{r}, \text{ and test if } \frac{\text{Tr}(\mathbf{H})^2}{\text{Det}(\mathbf{H})} > \frac{(10 + 1)^2}{10} = 12.1$$

(if **yes** discard the keypoint)

Detecting keypoints with DoG

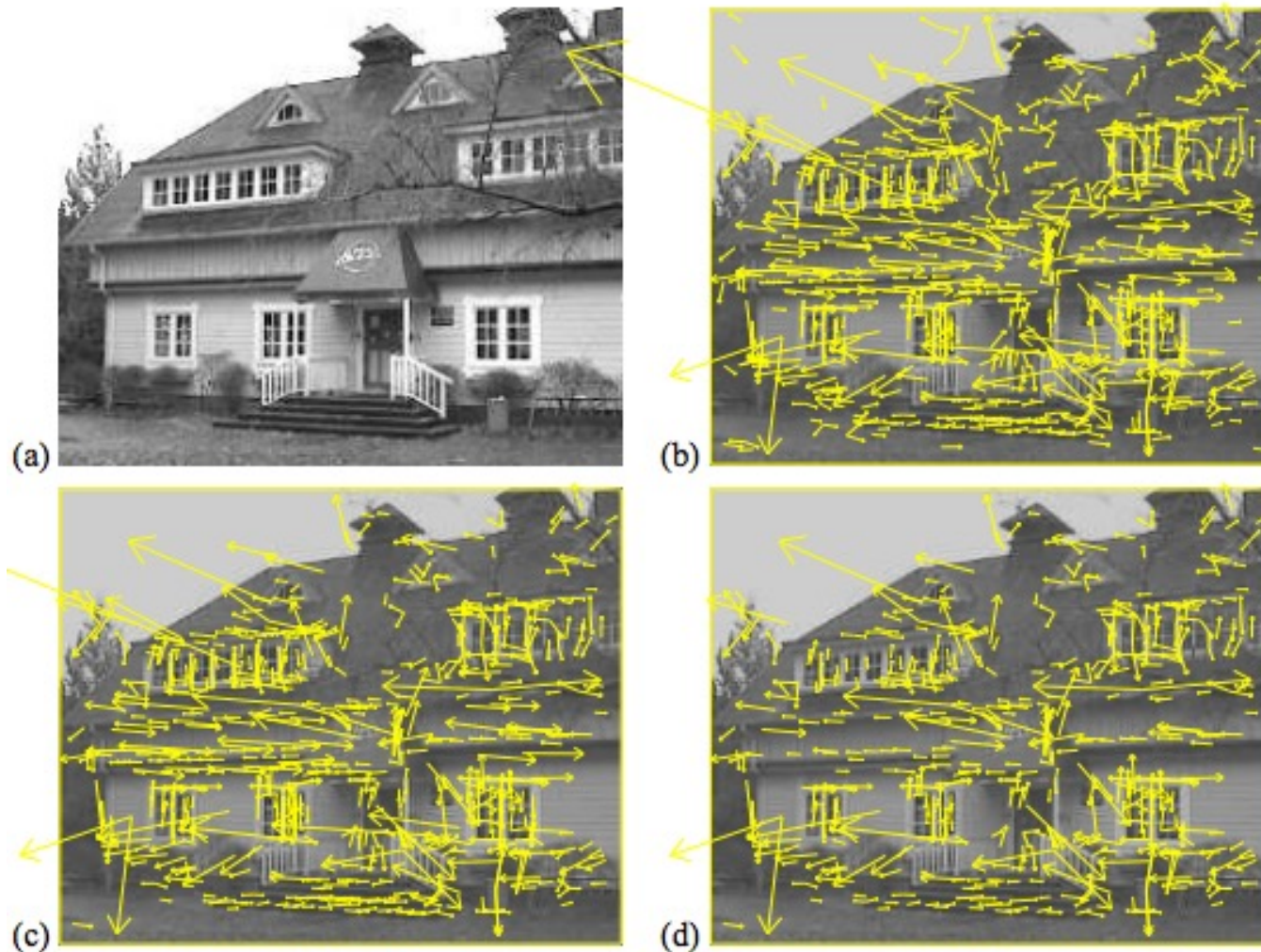


Figure 5: This figure shows the stages of keypoint selection. (a) The 233x189 pixel original image. (b) The initial 832 keypoints locations at maxima and minima of the difference-of-Gaussian function. Keypoints are displayed as vectors indicating scale, orientation, and location. (c) After applying a threshold on minimum contrast, 729 keypoints remain. (d) The final 536 keypoints that remain following an additional threshold on ratio of principal curvatures.

Major stages for computing SIFT features

1. **Scale-space extrema detection:** The first stage of computation searches over all scales and image locations. It is implemented efficiently by using a difference-of-Gaussian function to identify potential interest points that are invariant to scale and orientation.
2. **Keypoint localization:** At each candidate location, a detailed model is fit to determine location and scale. Keypoints are selected based on measures of their stability.
3. **Orientation assignment:** One or more orientations are assigned to each keypoint location based on local image gradient directions. All future operations are performed on image data that has been transformed relative to the assigned orientation, scale, and location for each feature, thereby providing invariance to these transformations.
4. **Keypoint descriptor:** The local image gradients are measured at the selected scale in the region around each keypoint. These are transformed into a representation that allows for significant levels of local shape distortion and change in illumination.

This approach has been named the Scale Invariant Feature Transform (SIFT), as it transforms image data into scale-invariant coordinates relative to local features.

Orientation assignment in SIFT

The orientation assignment stage gives SIFT features their rotation invariance. After detecting a keypoint in space and scale, we assign a dominant orientation to it so that the descriptor can be computed in a canonical direction, regardless of how the image is rotated.

Orientation assignment in SIFT

The orientation assignment stage gives SIFT features their rotation invariance. After detecting a keypoint in space and scale, we assign a dominant orientation to it so that the descriptor can be computed in a canonical direction, regardless of how the image is rotated.

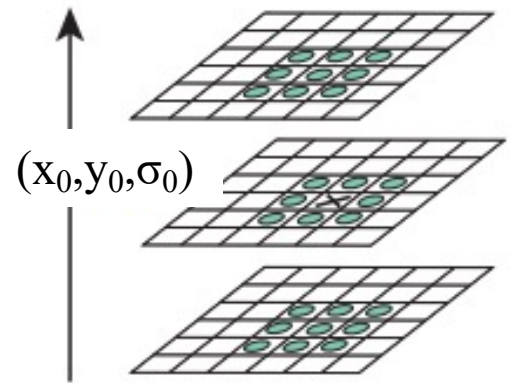
Assume a keypoint at location (x_0, y_0) and scale σ_0 .

1. Compute gradients in a neighborhood

- use the **Gaussian-blurred image** $L(x, y, \sigma_0)$ at the keypoint's scale (suppresses noise).
- for each pixel (x, y) in a local window (typically **16×16** around the keypoint) compute the **gradient magnitude** and **orientation**:

$$m(x, y) = \sqrt{(L(x + 1, y) - L(x - 1, y))^2 + (L(x, y + 1) - L(x, y - 1))^2}$$

$$\theta(x, y) = \tan^{-1}((L(x, y + 1) - L(x, y - 1)) / (L(x + 1, y) - L(x - 1, y)))$$



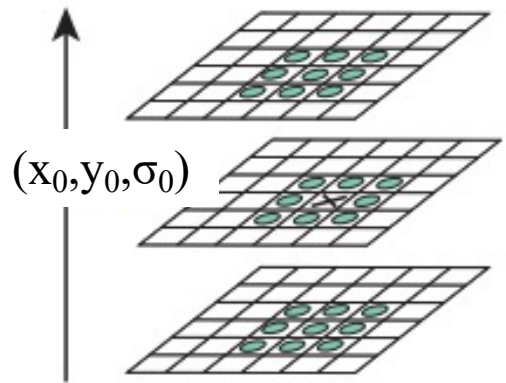
Orientation assignment in SIFT

The orientation assignment stage gives SIFT features their rotation invariance. After detecting a keypoint in space and scale, we assign a dominant orientation to it so that the descriptor can be computed in a canonical direction, regardless of how the image is rotated.

Assume a keypoint at location (x_0, y_0) and scale σ_0 .

1. Compute gradients in a neighborhood

- use the **Gaussian-blurred image** $L(x, y, \sigma_0)$ at the keypoint's scale (suppresses noise).
- for each pixel (x, y) in a local window (typically **16×16** around the keypoint) compute the **gradient magnitude** and **orientation**:

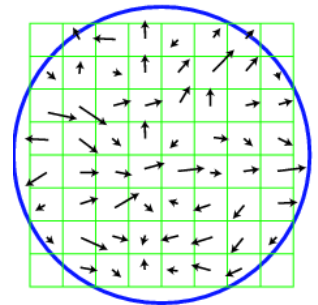


$$m(x, y) = \sqrt{(L(x + 1, y) - L(x - 1, y))^2 + (L(x, y + 1) - L(x, y - 1))^2}$$

$$\theta(x, y) = \tan^{-1}((L(x, y + 1) - L(x, y - 1)) / (L(x + 1, y) - L(x - 1, y)))$$

2. Weight gradients by a Gaussian

- apply a **Gaussian weighting** (with $\sigma = 1.5\sigma_0$) centered at the keypoint:
- gives more weight to gradients **closer to the keypoint**
- suppresses distant gradients that are likely noisier



Orientation assignment in SIFT

Assume a keypoint at location (x_0, y_0) and scale σ_0 .

1. **Compute gradients in a neighborhood**

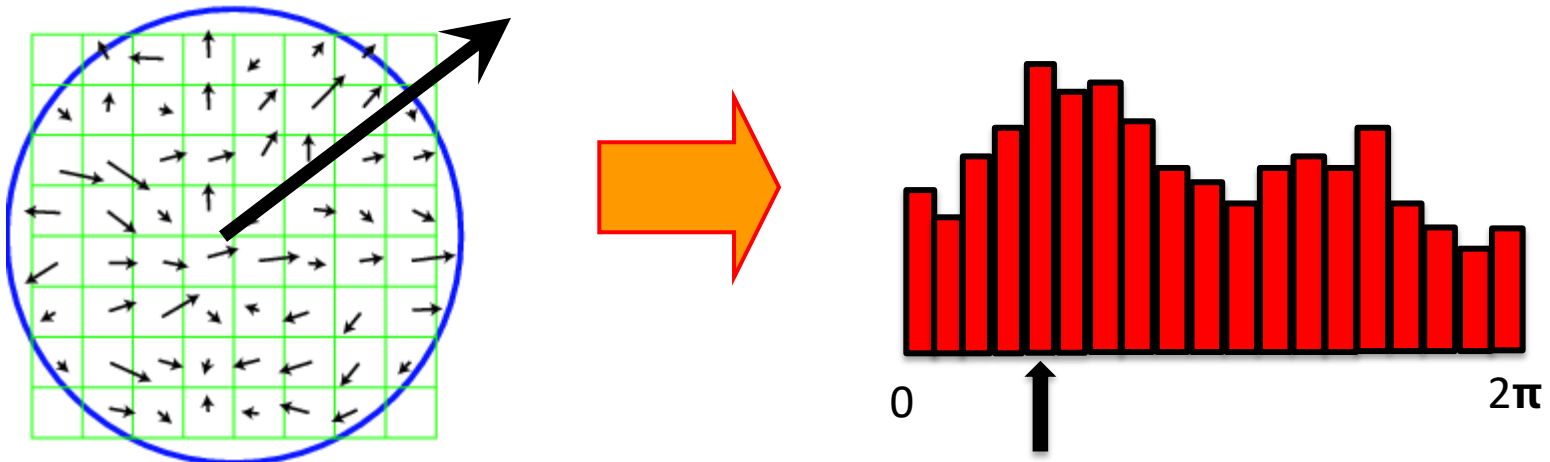
2. **Weight gradients by a Gaussian**

3. **Build an orientation histogram**

- histogram with **36 bins** (10° per bin over 360°)
- for each pixel in the neighborhood add its weighted **magnitude** to the histogram **bin** corresponding to its angle θ .

4. **Find the dominant orientation(s)**

- identify the **peak** of the histogram
- this peak orientation becomes the **main orientation** of the keypoint
- if there are **other local maxima within 80%** of the peak, **create additional keypoints** at the same location/scale but with different orientations



Major stages for computing SIFT features

1. **Scale-space extrema detection:** The first stage of computation searches over all scales and image locations. It is implemented efficiently by using a difference-of-Gaussian function to identify potential interest points that are invariant to scale and orientation.
2. **Keypoint localization:** At each candidate location, a detailed model is fit to determine location and scale. Keypoints are selected based on measures of their stability.
3. **Orientation assignment:** One or more orientations are assigned to each keypoint location based on local image gradient directions. All future operations are performed on image data that has been transformed relative to the assigned orientation, scale, and location for each feature, thereby providing invariance to these transformations.
4. **Keypoint descriptor:** The local image gradients are measured at the selected scale in the region around each keypoint. These are transformed into a representation that allows for significant levels of local shape distortion and change in illumination.

This approach has been named the Scale Invariant Feature Transform (SIFT), as it transforms image data into scale-invariant coordinates relative to local features.

SIFT descriptor

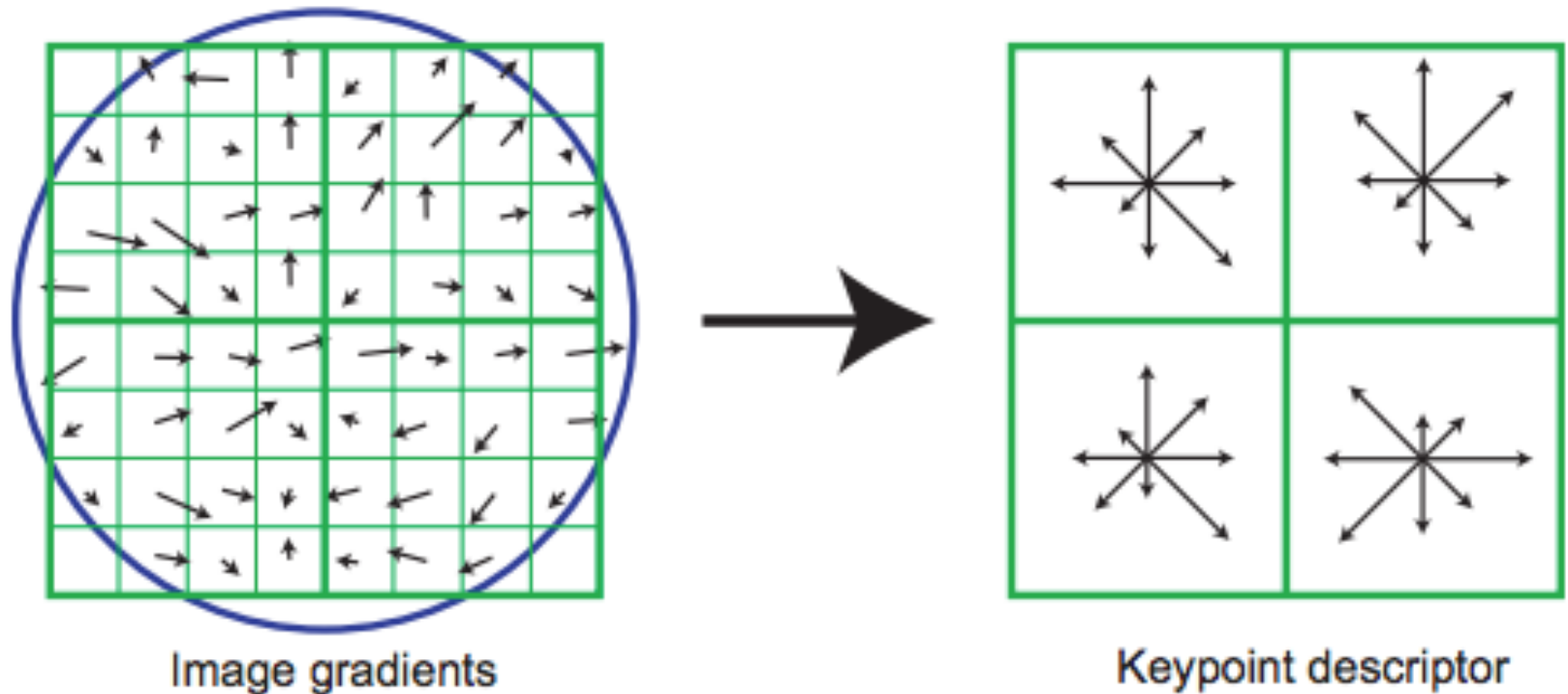


Figure 7: A keypoint descriptor is created by first computing the gradient magnitude and orientation at each image sample point in a region around the keypoint location, as shown on the left. These are weighted by a Gaussian window, indicated by the overlaid circle. These samples are then accumulated into orientation histograms summarizing the contents over 4x4 subregions, as shown on the right, with the length of each arrow corresponding to the sum of the gradient magnitudes near that direction within the region. This figure shows a 2x2 descriptor array computed from an 8x8 set of samples, whereas the experiments in this paper use 4x4 descriptors computed from a 16x16 sample array.

SIFT descriptor

Given a keypoint at location (x,y) , with a selected scale σ_0 and an orientation θ_0 , the descriptor captures the distribution of local image gradients relative to that orientation, in a region around the keypoint.

1. Take a window of 16×16 pixels centered on the keypoint from $L(x,y,\sigma)$.
2. The 16×16 patch is split into 16 subregions of 4×4 pixels each. These subregions form a 4×4 grid. Each subregion will contribute a histogram of local gradient directions.
3. Update the orientations $\theta(x,y)$ of each pixel relative to the main orientation θ_0 , such that $\theta_{\text{rel}}(x,y) = \theta(x,y) - \theta_0$.
4. Apply a Gaussian window (typically $\sigma=8$) centered on the keypoint to weight contributions by distance (to emphasize the center of the patch). Each pixel's magnitude is multiplied by this Gaussian weight.

SIFT descriptor

Given a keypoint at location (x,y) , with a selected scale σ_0 and an orientation θ_0 , the descriptor captures the distribution of local image gradients relative to that orientation, in a region around the keypoint.

5. Each 4×4 cell accumulates a histogram of 8 orientation bins ($0-360^\circ$). Each pixel votes into its bin based on its orientation (relative to keypoint) and Gaussian-weighted magnitude. Trilinear interpolation is used: vote across 2 spatial directions and orientation direction, to avoid hard bin edges.
6. Form the descriptor vector. We have: 4×4 cells $\times$ 8 orientation bins = 128 values.
7. Normalize the 128D vector to unit length to reduce sensitivity to illumination in three stages:

$$\mathbf{v} \leftarrow \frac{\mathbf{v}}{||\mathbf{v}||} \quad v_i = \min(v_i, 0.2) \quad \text{then} \quad \mathbf{v} \leftarrow \frac{\mathbf{v}}{||\mathbf{v}||}$$

Width and # of orientations

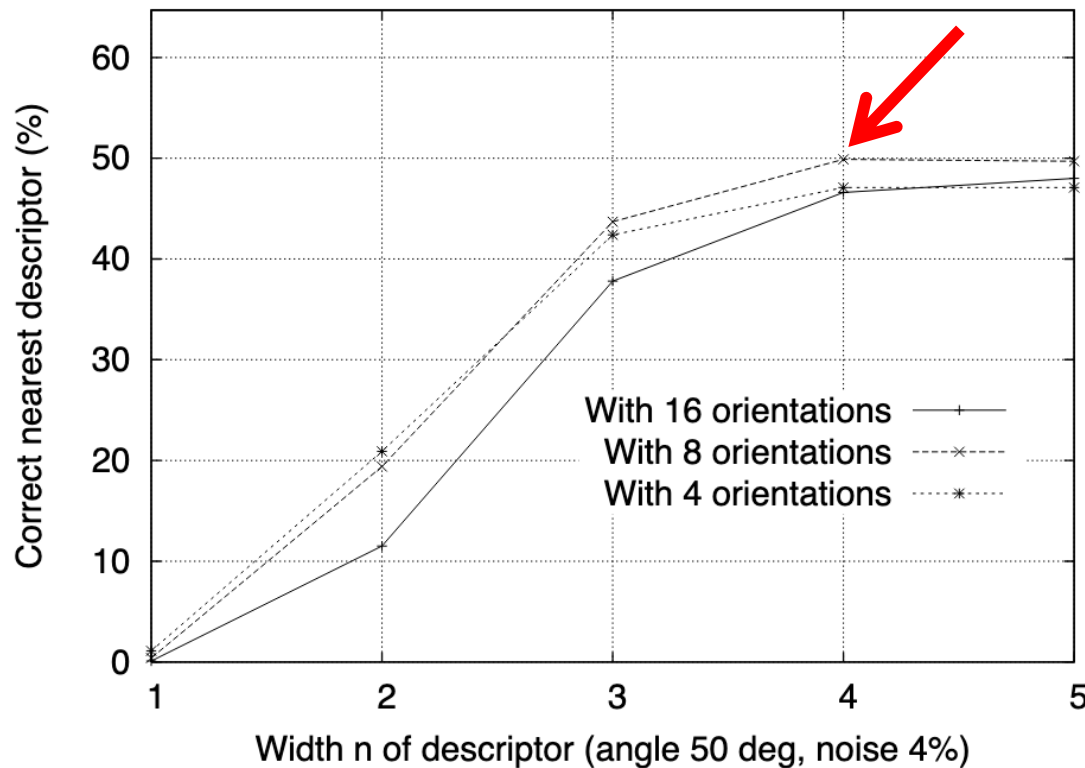


Figure 8: This graph shows the percent of keypoints giving the correct match to a database of 40,000 keypoints as a function of width of the $n \times n$ keypoint descriptor and the number of orientations in each histogram. The graph is computed for images with affine viewpoint change of 50 degrees and addition of 4% noise.

Stability to image noise

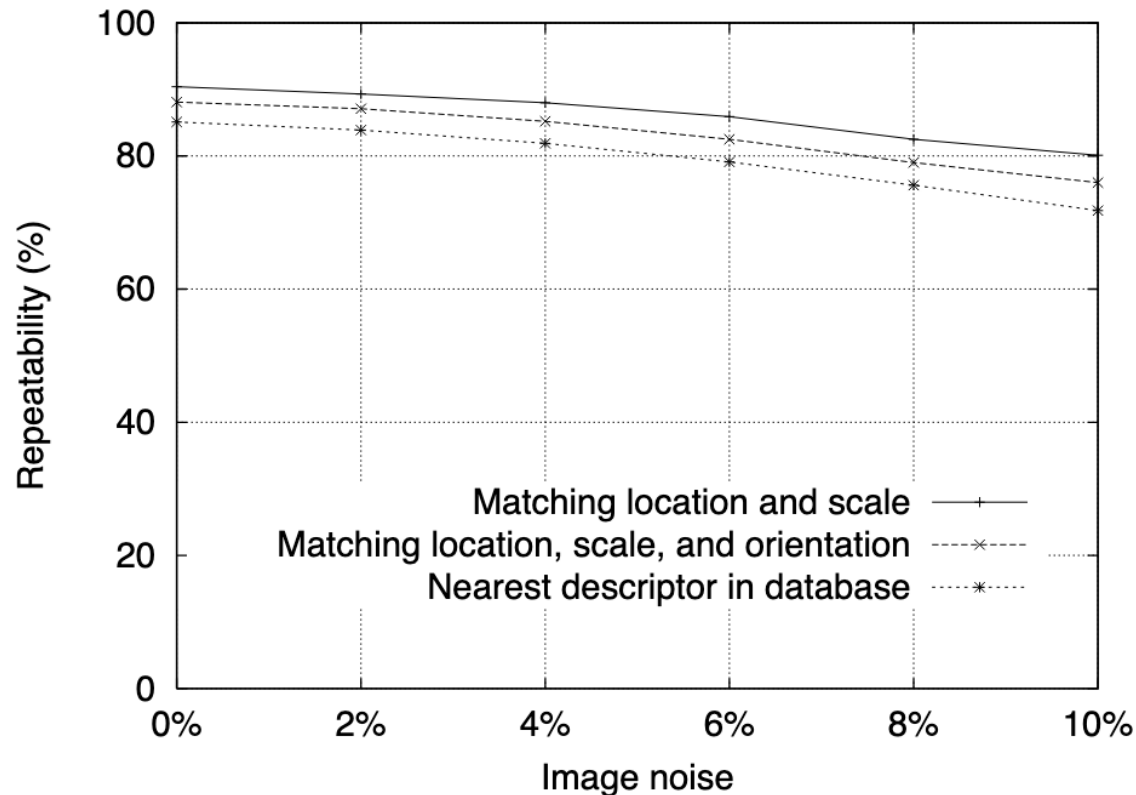


Figure 6: The top line in the graph shows the percent of keypoint locations and scales that are repeatably detected as a function of pixel noise. The second line shows the repeatability after also requiring agreement in orientation. The bottom line shows the final percent of descriptors correctly matched to a large database.

Stability to affine distortion

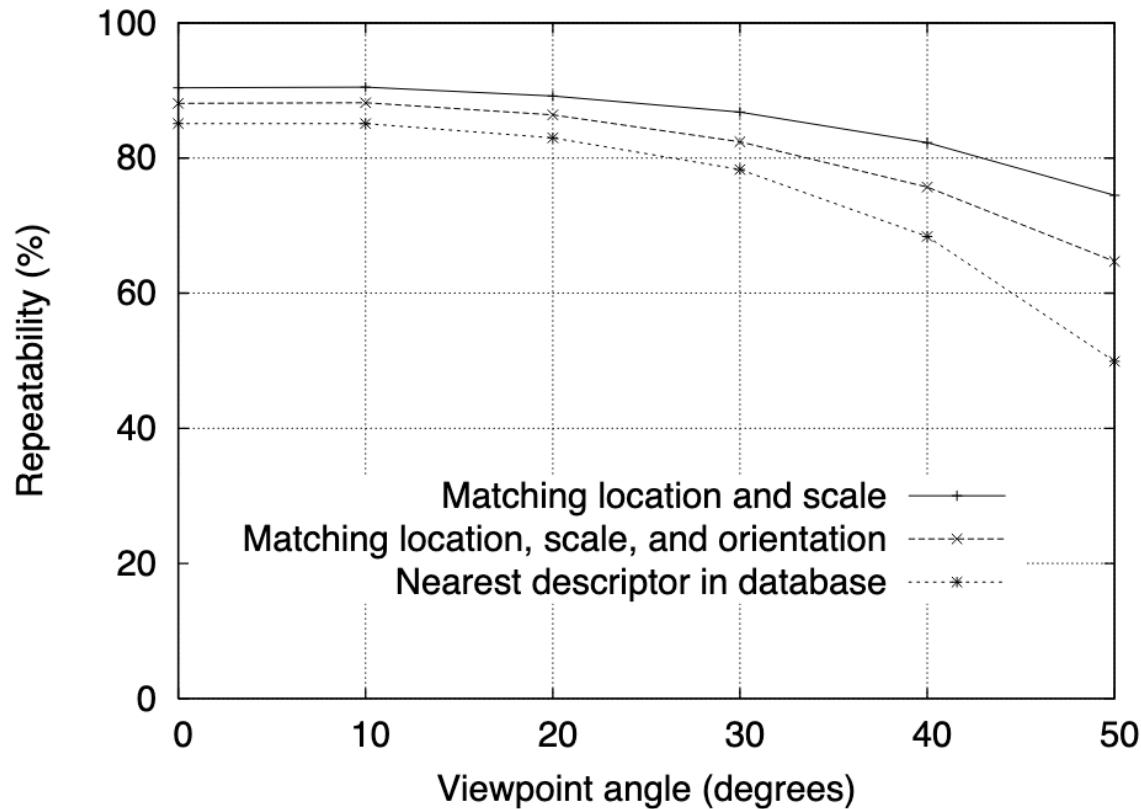


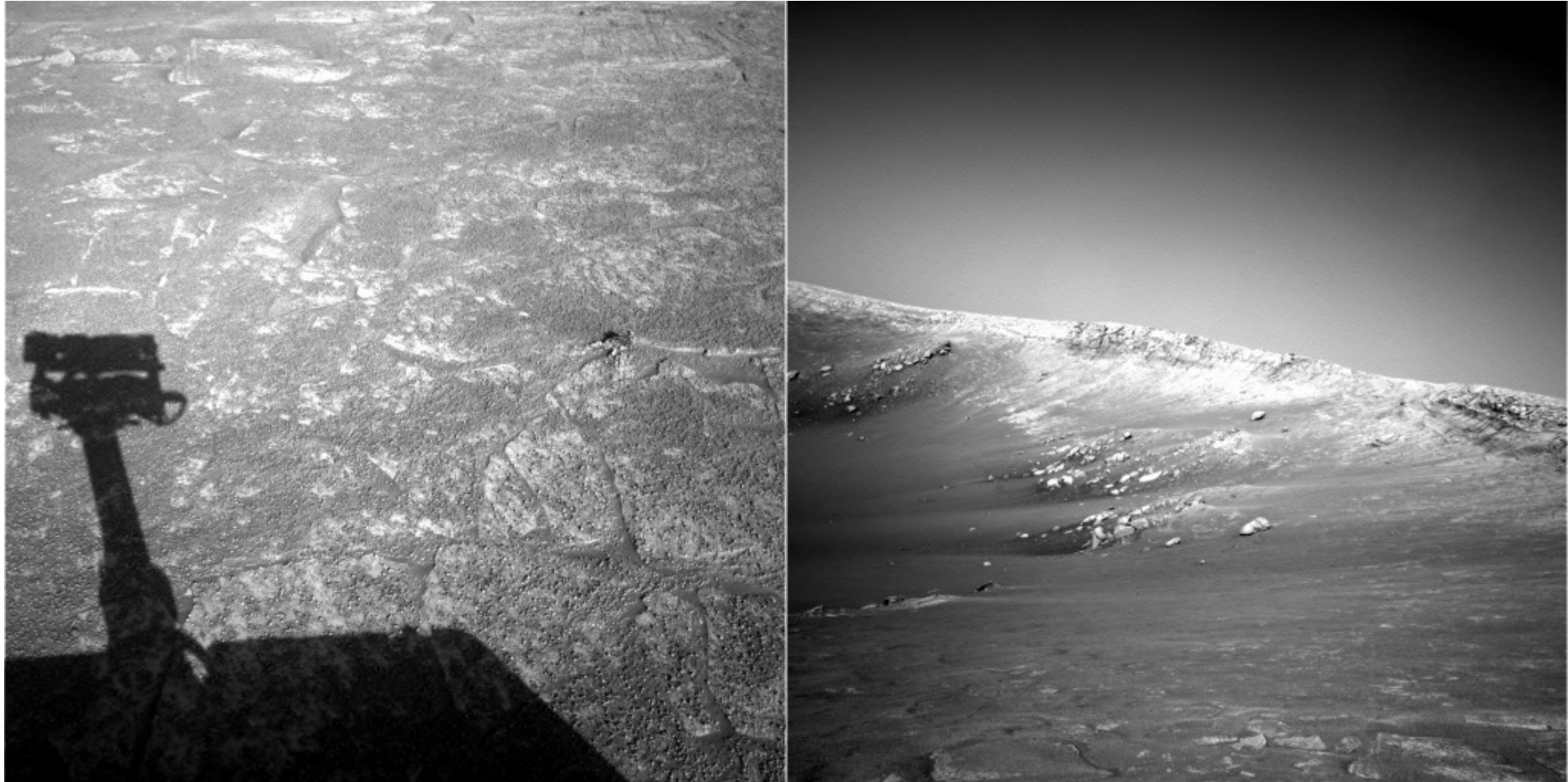
Figure 9: This graph shows the stability of detection for keypoint location, orientation, and final matching to a database as a function of affine distortion. The degree of affine distortion is expressed in terms of the equivalent viewpoint rotation in depth for a planar surface.

SIFT descriptor

- Extraordinarily robust matching technique
 - Can handle changes in viewpoint
 - Up to about 60 degree out of plane rotation
 - Can handle significant changes in illumination
 - Sometimes even day vs. night (below)
 - Fast and efficient—can run in real time
 - Lots of code available, e.g.
<http://www.vlfeat.org/overview/sift.html>

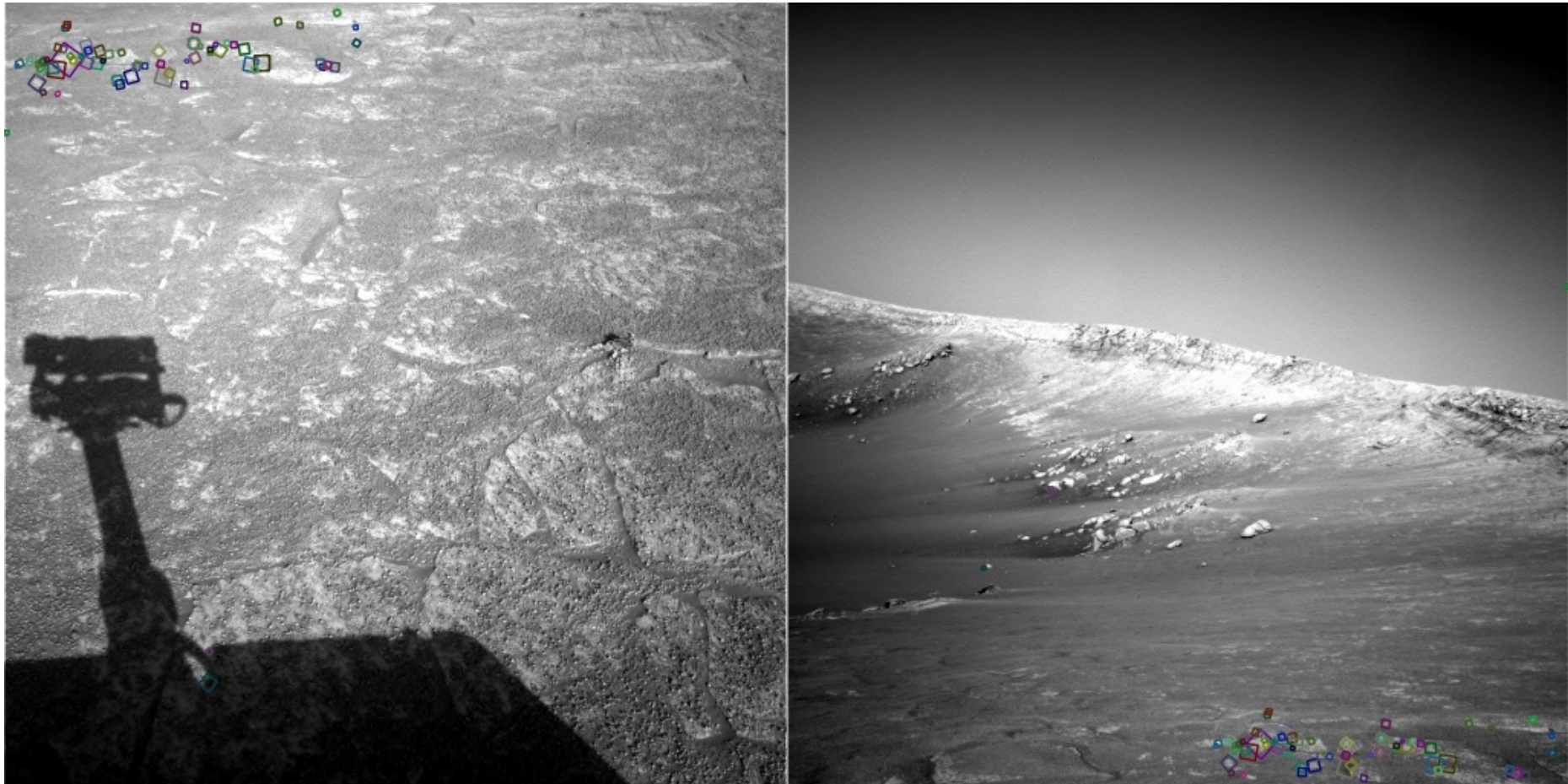


Example



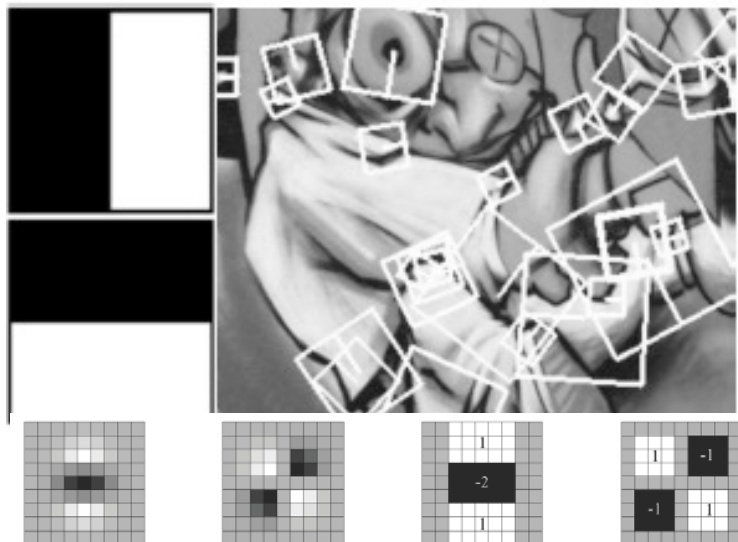
NASA Mars Rover images

Example



NASA Mars Rover images
with SIFT feature matches
Figure by Noah Snavely

Local Descriptors: SURF



Many other efficient descriptors
are also available

Fast approximation of SIFT idea

Efficient computation by 2D box filters &
integral images

⇒ 6 times faster than SIFT

Equivalent quality for object identification

GPU implementation available

Feature extraction @ 200Hz
(detector + descriptor, 640×480 img)

<http://www.vision.ee.ethz.ch/~surf>

Local Descriptors: ORB

- Many similarities to SIFT/SURF
- Designed for efficiency and robustness to orientation
- Not designed for scale robustness
- Used for tracking and long-range matching in ORB-SLAM

http://www.willowgarage.com/sites/default/files/orb_final.pdf (ICCV 2011)
<http://webdiis.unizar.es/~raulmur/orbslam/>

SuperPoint: Self-Supervised Interest Point Detection and Description

Daniel DeTone
Magic Leap
Sunnyvale, CA

ddetone@magicleap.com

Tomasz Malisiewicz
Magic Leap
Sunnyvale, CA

tmalisiewicz@magicleap.com

Andrew Rabinovich
Magic Leap
Sunnyvale, CA

arabinovich@magicleap.com

Abstract

This paper presents a self-supervised framework for training interest point detectors and descriptors suitable for a large number of multiple-view geometry problems in computer vision. As opposed to patch-based neural networks, our fully-convolutional model operates on full-sized images and jointly computes pixel-level interest point locations and associated descriptors in one forward pass. We introduce Homographic Adaptation, a multi-scale, multi-homography approach for boosting interest point detection repeatability and performing cross-domain adaptation (e.g., synthetic-to-real). Our model, when trained on the MS-COCO generic image dataset using Homographic Adaptation, is able to repeatedly detect a much richer set of interest points than the initial pre-adapted deep model and any other traditional corner detector. The final system gives rise to state-of-the-art homography estimation results on HPatches when compared to LIFT, SIFT and ORB.

1. Introduction

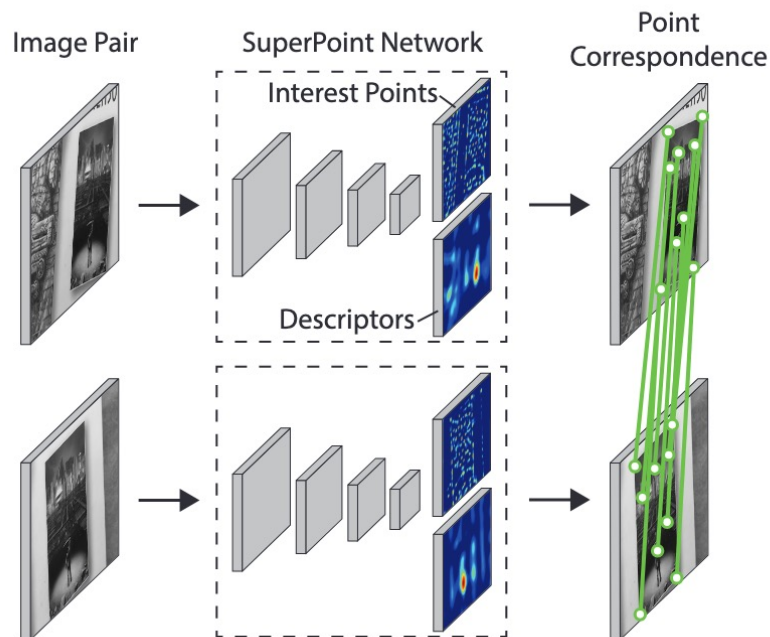


Figure 1. **SuperPoint for Geometric Correspondences.** We present a fully-convolutional neural network that computes SIFT-like 2D interest point locations and descriptors in a single forward pass and runs at 70 FPS on 480×640 images with a Titan X GPU.

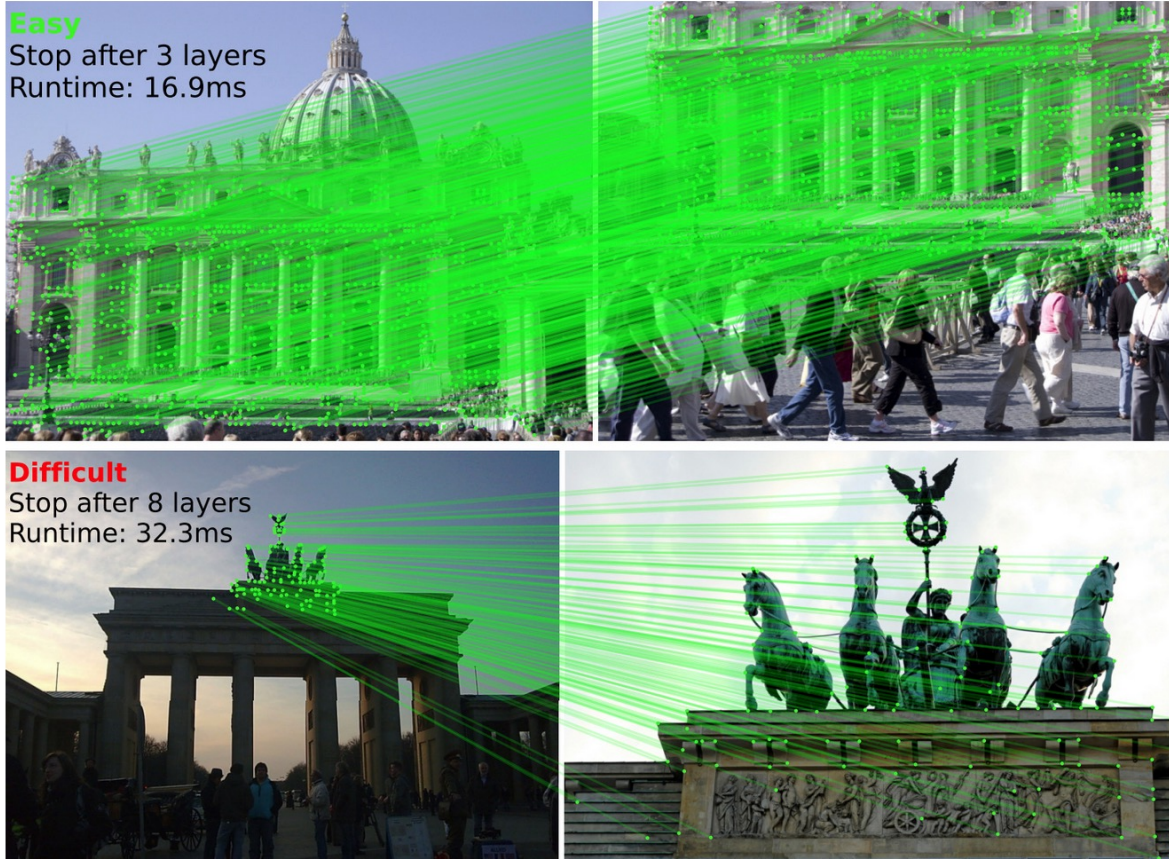
LightGlue ⚡

Local Feature Matching at Light Speed

[Philipp Lindenberger](#) · [Paul-Edouard Sarlin](#) · [Marc Pollefeys](#)

ICCV 2023

[Paper](#) | [Colab](#) | [Poster](#) | [Train your own!](#)



LightGlue is a deep neural network that matches sparse local features across image pairs. An adaptive mechanism makes it fast for easy pairs (top) and reduces the computational complexity for difficult ones (bottom).

Exam – evaluation in June

$$\text{Grade} = \min(10, P1 + P2 + \text{bonus})$$

- $P1 = \text{project 1} = 5 \text{ points (+ some bonus?)}$
- $P2 = \text{project 2} = 5 \text{ points (+ some bonus?)}$

Project 1 – good to know

- 5 points + 0.5 bonus points = 5.5p maximum
- deadline: in ~ 4 weeks time, Sunday, 3th of May 2026, 23:59
 - **hard deadline – no late submission policy**
 - upload your source code
 - on Monday, 4th of May 2026 we will release the test set. You will run your (untouched) code on test images and send us your results. Immediately we will published the intermediate grades (based on your results). Final grades for Project 1 will be published after carefully checking your code, your pdf report and listening to your oral presentation (~2 weeks time).
- do not share/copy the solution with/from your colleagues: you + your colleague/s will get 0 points

Project 1: Ingenious Score Calculator



Ingenious

Ingenious is a strategic, abstract tile-placement game that challenges players to build connections and maximize scoring across multiple colors simultaneously. The game uses hexagonal tiles, each featuring two colored segments chosen from six possible colors (see Figure 1). These tiles are placed on a hexagonal grid board, where players aim to match colors by extending lines outward from the edges of newly placed tiles.

Each placement scores points for every continuous line of matching colors connected to the tile, making positioning and foresight essential. Unlike many games, the objective is not simply to achieve the highest total score, but to develop a balanced score across all six colors - since a player's final score is determined by their lowest-scoring color.

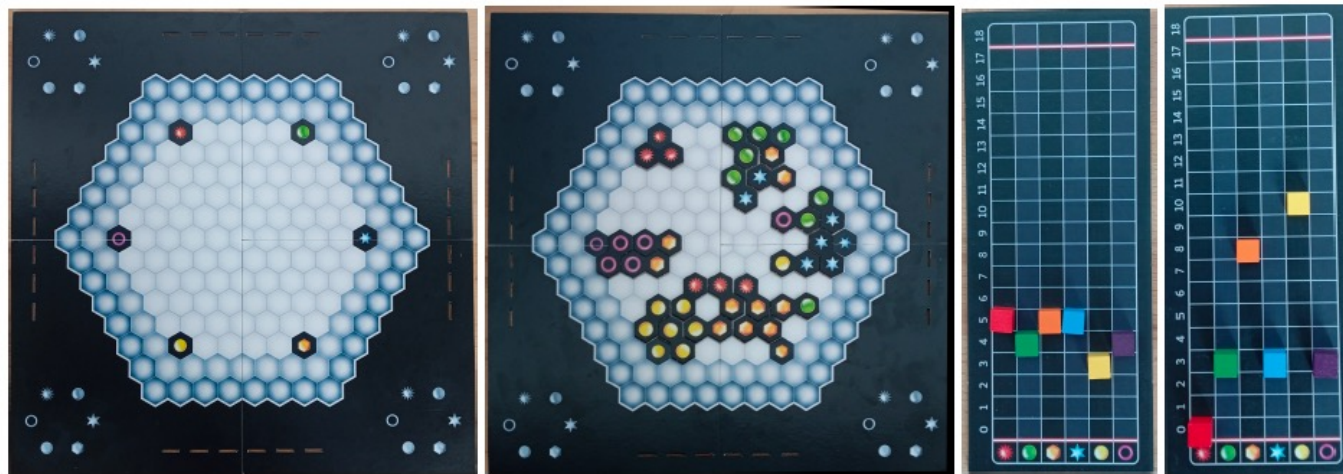


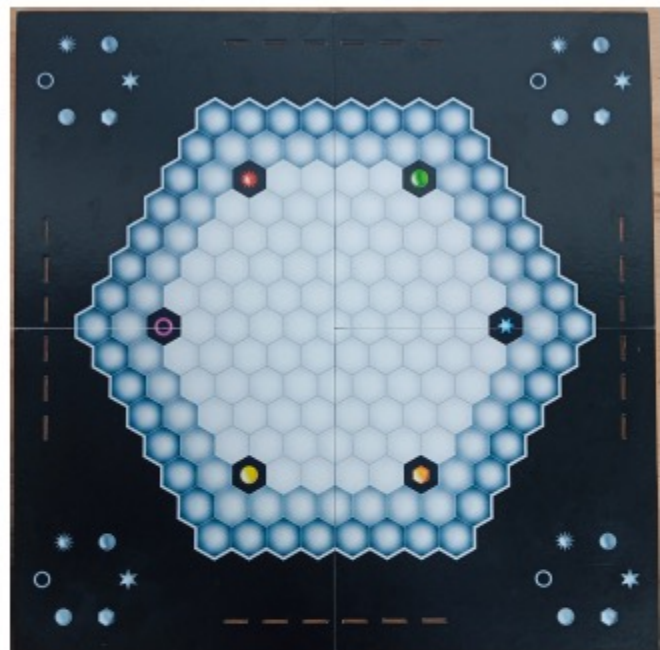
Figure 1: Illustration of the *Ingenious* game. **Left:** initial board configuration, consisting of an empty hexagonal grid with pre-printed colored symbols. **Middle:** board state after 10 moves made by each of the two players (20 moves in total). **Right:** table scores of the two players after 10 moves each, showing the accumulated score for each color.

Board

The game of *Ingenious* is played on a fixed board made of hexagonal cells arranged in a symmetric hexagonal layout. Each cell represents a valid position that may be occupied by one segment of a tile.

The board is initially empty, except for a small number of colored symbols printed on specific cells (see Figure 1 Left). These symbols correspond to the six colors (red, green, orange, blue, yellow and purple) used in the game and are relevant for the initial placement and for score computation.

The geometric structure of the board is essential for score computation, since points are awarded by counting contiguous sequences of matching colors along straight directions of the hexagonal grid.

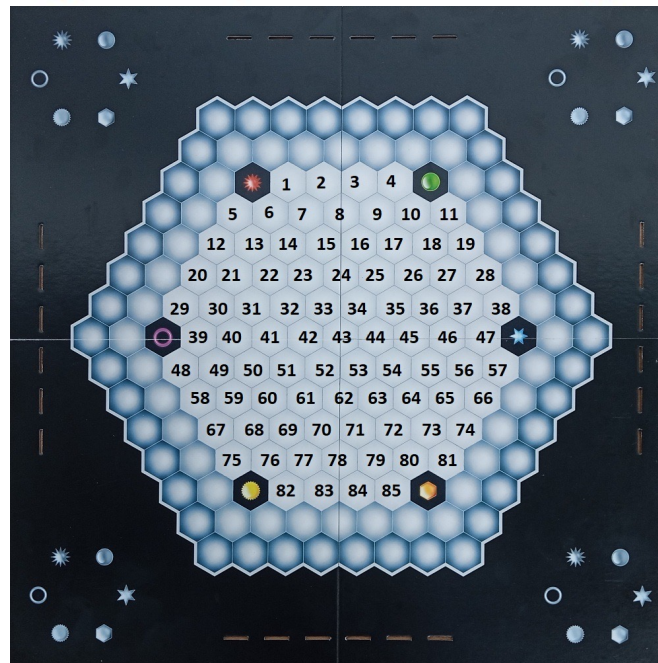


Board – for two players

The game of *Ingenious* is played on a fixed board made of hexagonal cells arranged in a symmetric hexagonal layout. Each cell represents a valid position that may be occupied by one segment of a tile.

The board is initially empty, except for a small number of colored symbols printed on specific cells (see Figure 1 Left). These symbols correspond to the six colors (red, green, orange, blue, yellow and purple) used in the game and are relevant for the initial placement and for score computation.

The geometric structure of the board is essential for score computation, since points are awarded by counting contiguous sequences of matching colors along straight directions of the hexagonal grid.



Tiles

Players interact with the board using *tiles*, each composed of two adjacent hexagonal segments. Every segment is assigned one of six possible colors, and thus each tile represents a pair of colors.

Tiles in *Ingenious* are defined by *unordered* pairs of colors. This means that a tile containing colors (**red**, **blue**) is equivalent to (**blue**, **red**). Both identical-color tiles (e.g., **red-red**) and mixed-color tiles (e.g., **red-blue**) are part of the game.

The set of tiles used during gameplay has the following properties:

- six distinct colors: **red**, **green**, **orange**, **blue**, **yellow** and **purple**;
- each tile consists of exactly two adjacent hexagonal segments;
- tiles correspond to all possible unordered pairs of colors, including identical pairs;
- multiple instances of each color pair are available, forming the complete pool of tiles.

Each segment of a placed tile occupies one cell on the board, and the two segments are always positioned on adjacent cells, reflecting the geometric structure of the board. All possible tile types, corresponding to all unordered pairs of colors, are illustrated in Figure 2, where each combination is shown explicitly.

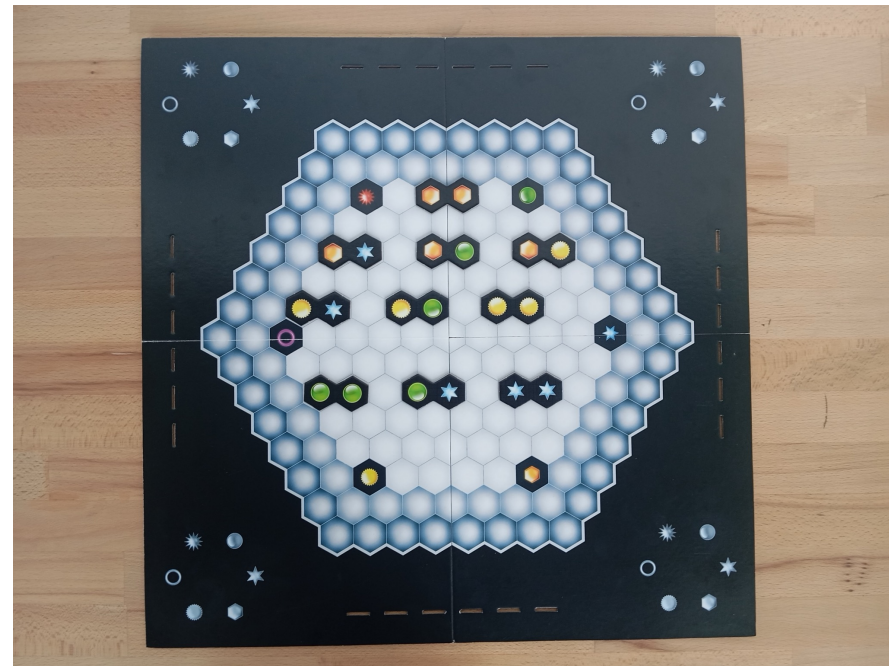
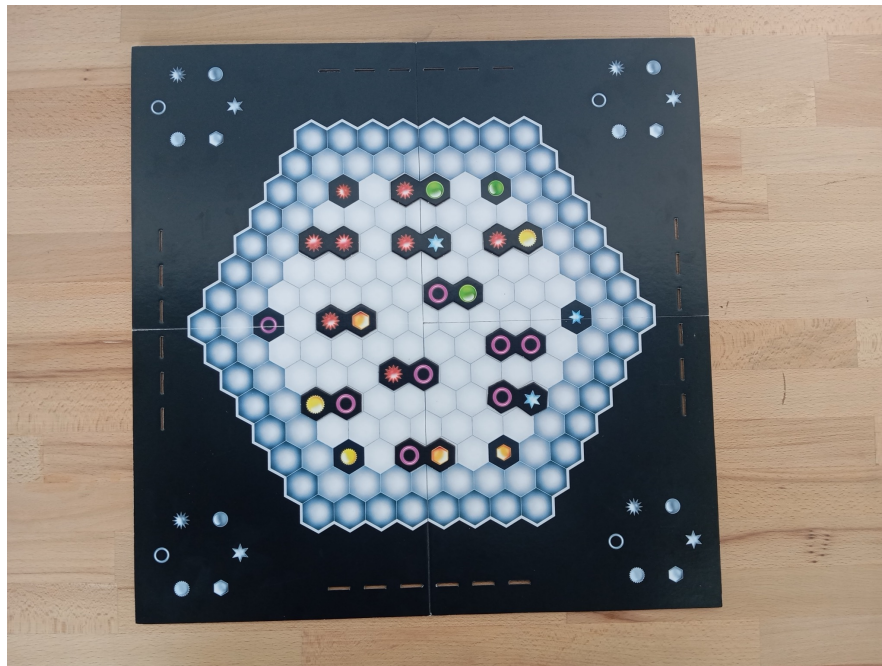
Tiles

Players interact with the board using *tiles*, each composed of two adjacent hexagonal segments. Every segment is assigned one of six possible colors, and thus each tile represents a pair of colors.

Tiles in *Ingenious* are defined by *unordered* pairs of colors. This means that a tile containing colors (**red**, **blue**) is equivalent to (**blue**, **red**). Both identical-color tiles (e.g., **red-red**) and mixed-color tiles (e.g., **red-blue**) are part of the game.

The set of tiles used during gameplay has the following properties:

- six distinct colors: **red**, **green**, **orange**, **blue**, **yellow** and **purple**;



Piece placement rules

At each turn, a player selects one tile and places it on the board. A tile always occupies exactly two adjacent hexagonal cells.

In this project, we consider the variant of the game with **two players**. In this setting, tiles may be placed only on the *white (interior)* cells of the board, i.e., cells that do not contain pre-printed colored symbols. These valid placement cells are illustrated in Figure 3.

A placement is considered valid if the tile occupies two *empty adjacent* white cells. Additional rules apply depending on the stage of the game:

- **First placement constraint:** At the beginning of the game, the first tile must be placed such that at least one of its segments is adjacent to a pre-existing colored symbol printed on the board that has no neighboring tiles.
- **Subsequent placements:** Starting from the second move, tiles may be placed on any pair of empty adjacent white cells on the board. There is no requirement for the tile to be adjacent to previously placed tiles or to match any neighboring colors.

Important: Tile placement does not require connectivity to existing tiles. It is therefore possible to create isolated groups of tiles or leave gaps between placed pieces.

A player must place a tile at each turn, even if the placement does not immediately generate any points.

Gameplay

In this project, we consider the two-player variant of the *Ingenious* game. The two players take turns placing tiles on the board, alternating moves throughout the game.

At each turn, the active player performs the following steps:

- a) **Place a tile.** The player selects one tile from their current set (initially it has six tiles) and places it on the board according to the placement rules described previously.
- b) **Compute the score.** The player scores points based on the placement of the tile. The scoring is performed independently for each of the two colored segments of the tile.
- c) **Update the table score.** The player advances their markers on the table score for the corresponding colors.
- d) **Draw new tiles.** The player draws tiles from the bag until their set is replenished to the maximum number of tiles (six tiles).

Players must place a tile at each turn, even if the placement does not generate any points. The game proceeds with players alternating turns until no further valid placements are possible or the tile pool is exhausted.

Scoring

After placing a tile, the player scores points for each of the two colored segments of the tile, independently. For a given colored segment, the score is computed as follows:

- Consider the cell occupied by the segment and identify the six directions of the hexagonal grid.
- For each direction, examine the adjacent cell in that direction:
 - If the adjacent cell contains a symbol of the same color, continue moving in that direction along a straight line.
 - Count the number of consecutive cells containing the same color.
 - Stop when an empty cell or a cell with a different color is encountered.
- The number of consecutive matching cells in that direction represents the score contribution for that direction.

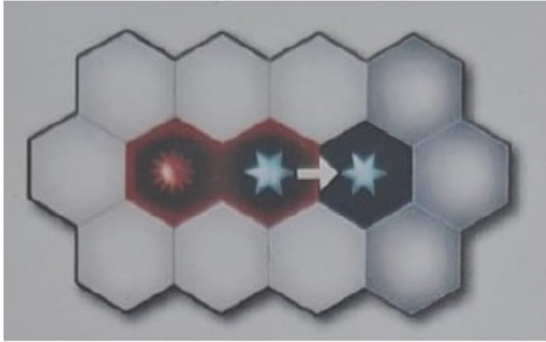
The total score for a colored segment is obtained by summing the contributions over all valid directions.

Important rules:

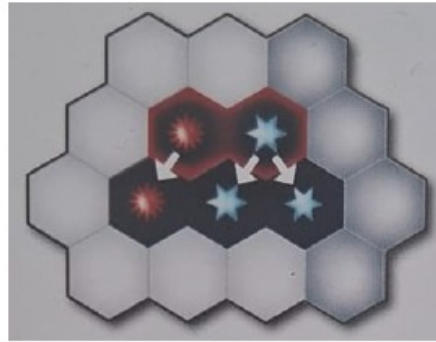
- Only cells that were already occupied before the current move (including pre-printed colored symbols) are considered when counting matches.
- The two segments of the same tile do not contribute to each other's score.
- If no adjacent cell in any direction contains a matching color, the score for that segment is zero.

The total score obtained in a turn is the sum of the scores computed for the two colored segments of the placed tile.

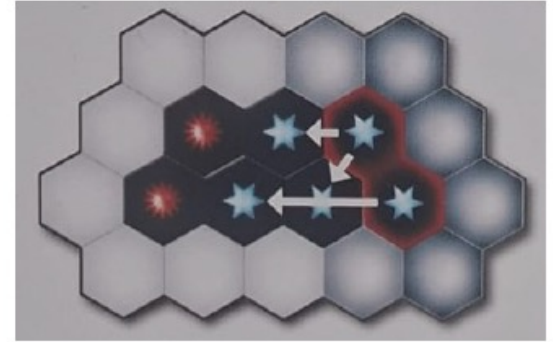
Scoring examples



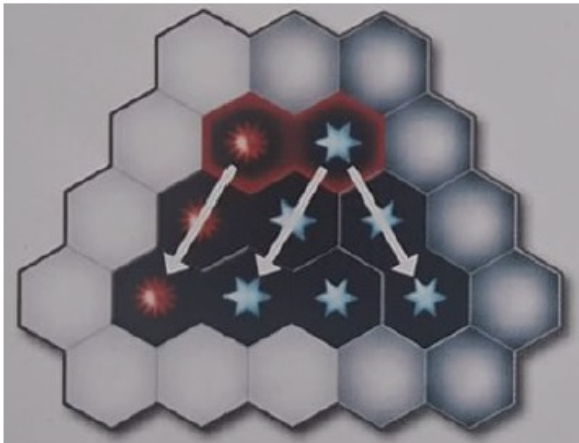
Example 1



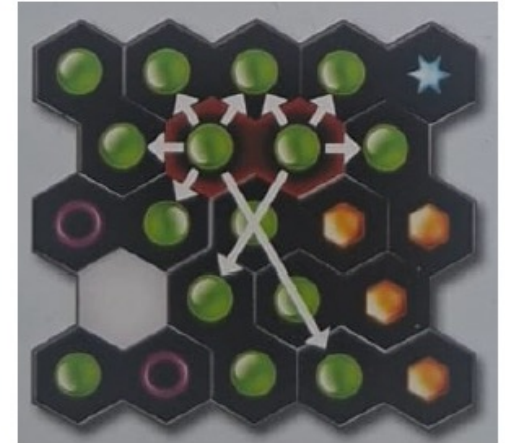
Example 2



Example 3



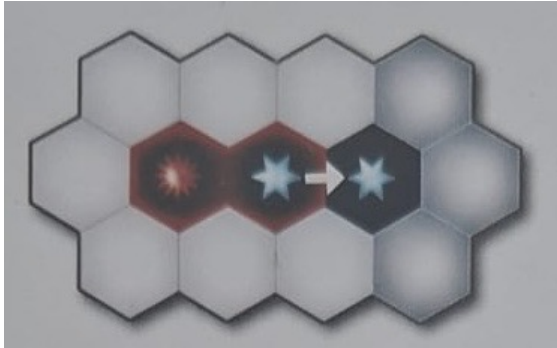
Example 4



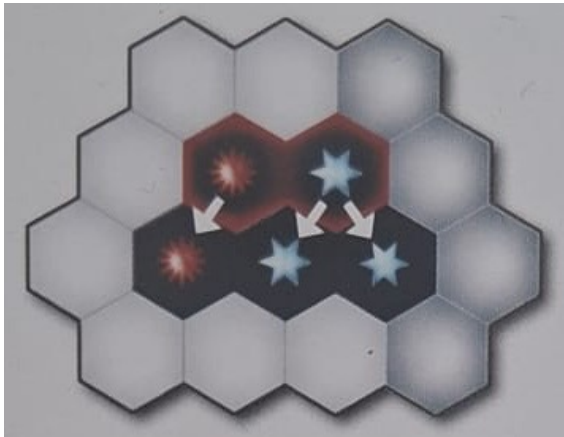
Example 5

Figure 4: Examples illustrating the scoring mechanism in *Ingenious*. We detail in the main text how points are computed for a tile placement in each of the 5 examples based on contiguous sequences of matching colors.

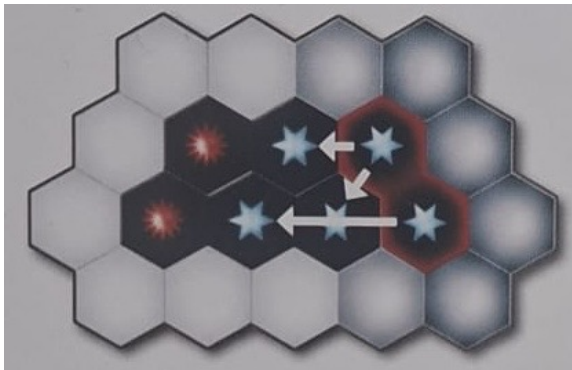
Scoring



Example 1. A tile containing a red and a blue symbol is placed. The red segment is not adjacent to any matching color, thus it scores 0 points. The blue segment is adjacent to a blue symbol already printed on the board, resulting in 1 point.

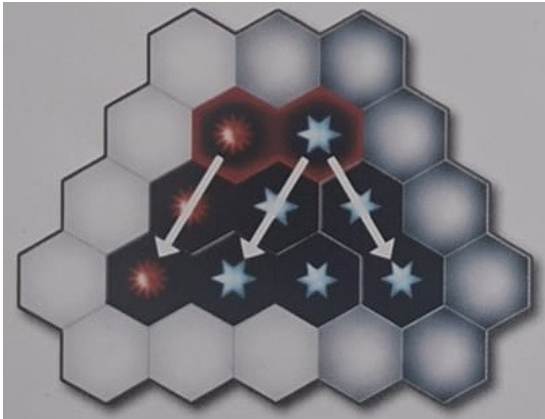


Example 2. A tile with a red and a blue symbol is placed. The red segment is adjacent to one red symbol, resulting in 1 point. The blue segment is adjacent to blue symbols in two different directions, resulting in 2 points.

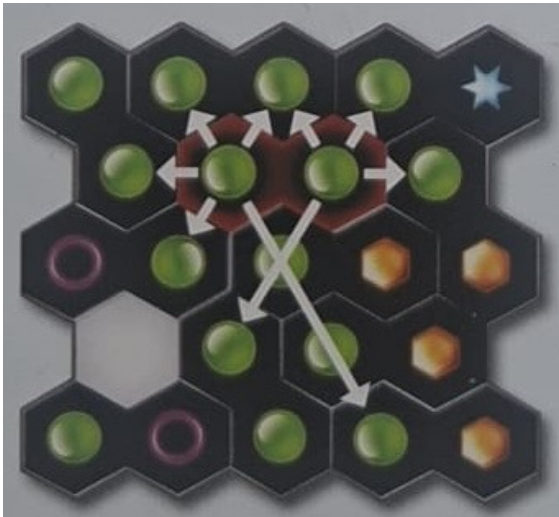


Example 3. A tile with two blue symbols is placed. One blue segment connects to a sequence of matching blue symbols extending in multiple directions, while the other contributes only along valid straight directions. In total, the placement results in 4 points for blue.

Scoring



Example 4. A tile with a red and a blue symbol is placed. The red segment forms a straight line of two matching red symbols, resulting in 2 points. The blue segment forms two separate straight sequences of blue symbols, resulting in a total of 4 points for blue.



Example 5. A tile with two green symbols is placed in a region surrounded by existing tiles. Multiple directions contain contiguous green symbols. By summing all valid straight-line contributions, the placement yields a total of 12 points for green.

Examples of scoring



(move 1)



(move 2)



(move 3)



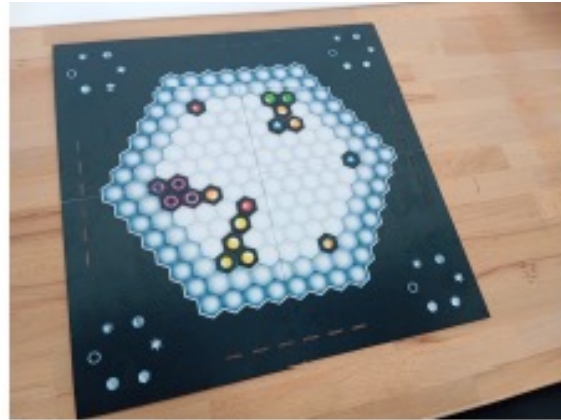
(move 4)

Figure 5: Illustration of eight consecutive moves performed by the two players. The detailed computation of the corresponding scores for each player after every move is provided in the main text.

Examples of scoring



(move 5)



(move 6)



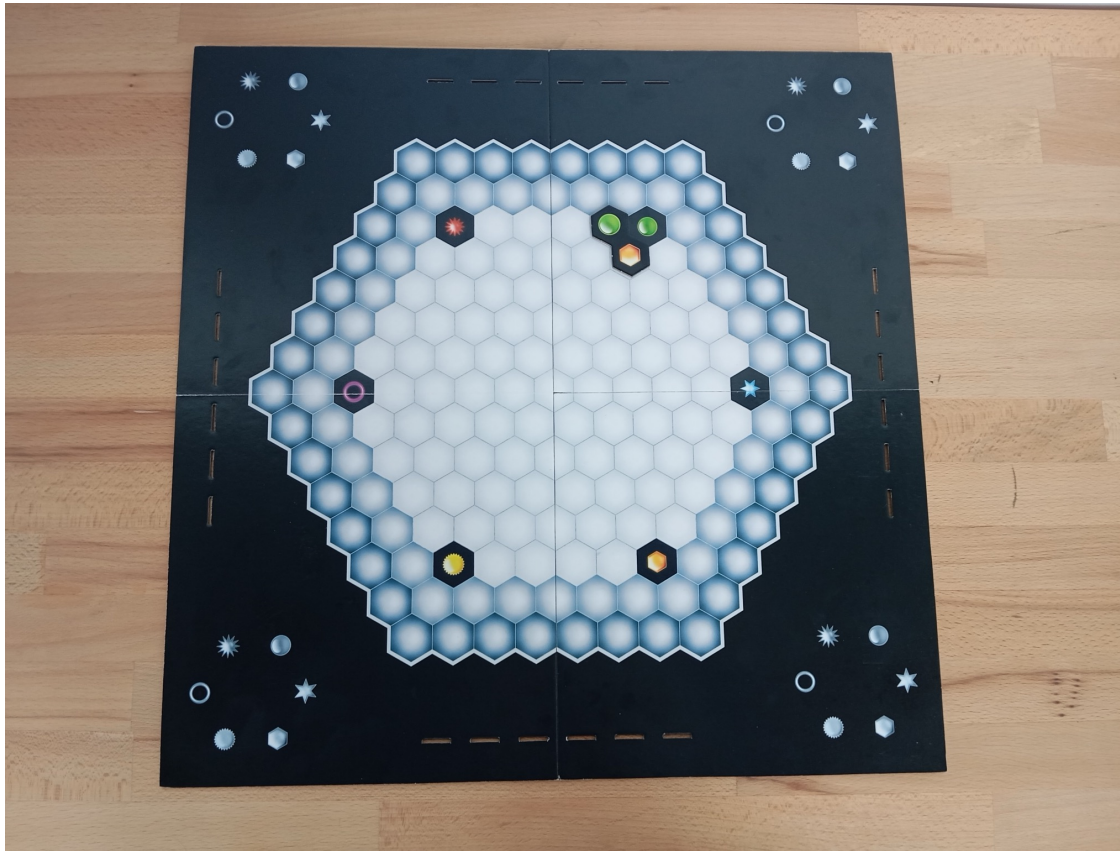
(move 7)



(move 8)

Figure 5: Illustration of eight consecutive moves performed by the two players. The detailed computation of the corresponding scores for each player after every move is provided in the main text.

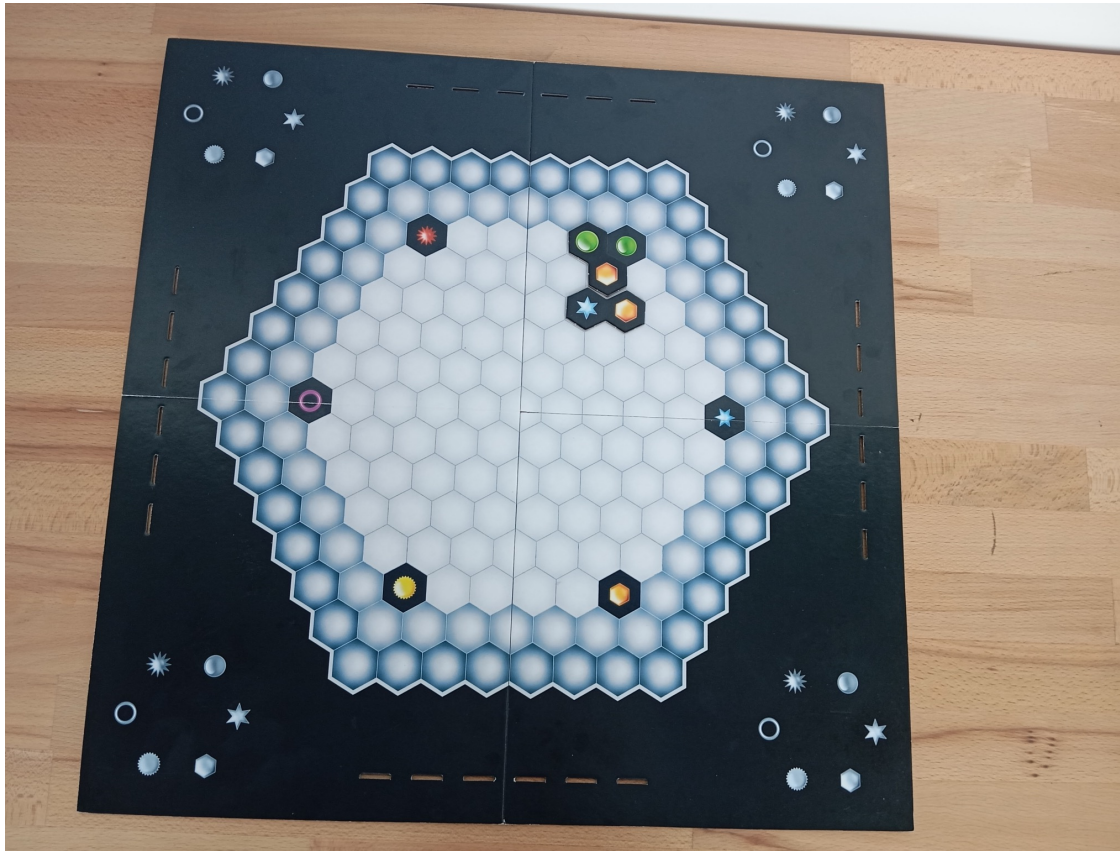
Examples of scoring



Move 1. Player 1 places a tile with a green segment at position 4 and an orange segment at position 10 (see Figure 3 for the indexing of board positions). The green segment is adjacent to a matching green symbol, yielding 1 point, while the orange segment does not contribute to the score. The resulting score table for player 1 after this move is:

$$t_1 = (\text{red} = 0, \text{green} = 1, \text{orange} = 0, \text{blue} = 0, \text{yellow} = 0, \text{purple} = 0) = (0, 1, 0, 0, 0, 0).$$

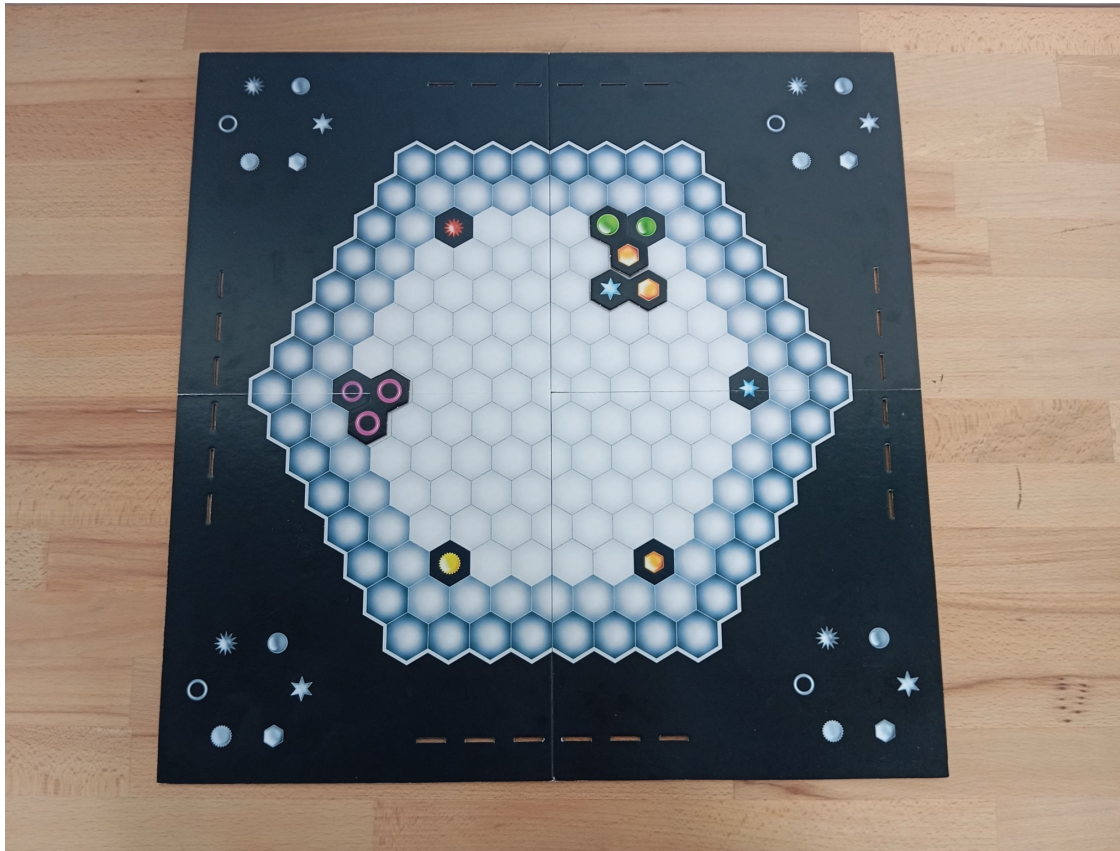
Examples of scoring



Move 2. Player 2 places a tile with a blue segment at position 17 and an orange segment at position 18. The orange segment is adjacent to a matching orange symbol, yielding 1 point, while the blue segment does not contribute to the score. The resulting score table for player 2 after this move is:

$$t_2 = (0, 0, 1, 0, 0, 0).$$

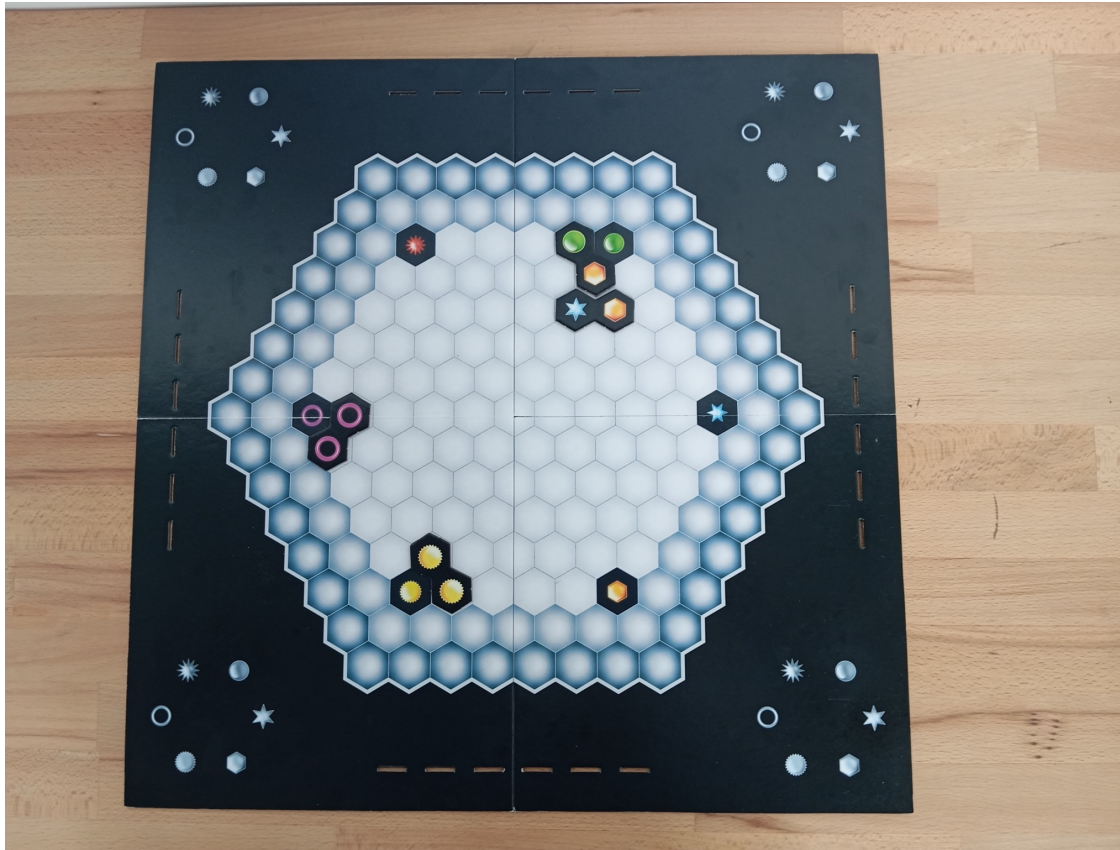
Examples of scoring



Move 3. Player 1 places a tile with both segments of color purple at positions 39 and 48. Each segment is adjacent to a matching purple symbol; therefore, each contributes 1 point, for a total of 2 points for purple. The resulting score table for player 1 after this move is:

$$t_1 = (0, 1, 0, 0, 0, 2).$$

Examples of scoring



Move 4. Player 2 places a tile with both segments of color yellow at positions 76 and 82. Each segment is adjacent to a matching yellow symbol; therefore, each contributes 1 point, for a total of 2 points for yellow. The resulting score table for player 2 after this move is:

$$t_2 = (0, 0, 1, 0, 2, 0).$$

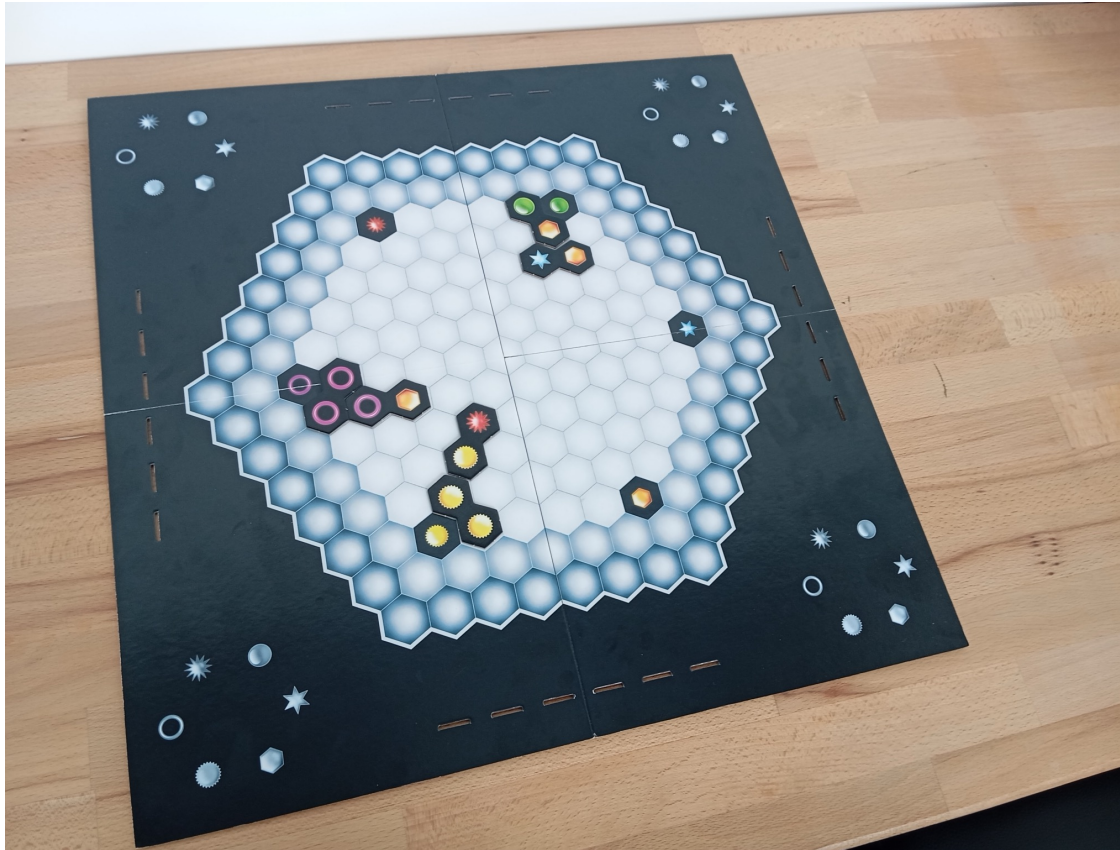
Examples of scoring



Move 5. Player 1 places a tile with a purple segment at position 49 and an orange segment at position 50. The purple segment is adjacent to two matching purple symbols (along two directions), yielding 2 points, while the orange segment does not contribute to the score. The resulting score table for player 1 after this move is:

$$t_1 = (0, 1, 0, 0, 0, 4).$$

Examples of scoring



Move 6. Player 2 places a tile with a red segment at position 61 and a yellow segment at position 69. The yellow segment is adjacent to a line of two matching yellow symbols, yielding 2 points, while the red segment does not contribute to the score. The resulting score table for player 2 after this move is:

$$t_2 = (0, 0, 1, 0, 4, 0).$$

Examples of scoring



Move 7. Player 1 places a tile with both segments of color green at positions 3 and 9. One green segment is adjacent to a line of two matching green symbols, while the other segment is adjacent to a single matching green symbol, yielding a total of 3 points for green. The resulting score table for player 1 after this move is:

$$t_1 = (0, 4, 0, 0, 0, 4).$$

Examples of scoring



Move 8. Player 2 places a tile with a green segment at position 16 and a blue segment at position 25. The green segment yields 2 points from a line of two matching green symbols, while the blue segment yields 1 point from a single matching blue symbol. The resulting score table for player 2 after this move is:

$$t_2 = (0, 2, 1, 1, 4, 0).$$

Corresponding table scores



(score table for move 1)



(score table for move 2)



(score table for move 3)



(score table for move 4)

Corresponding table scores



(score table for move 5)



(score table for move 6)



(score table for move 7)



(score table for move 8)

Data description

The released dataset (available at <https://tinyurl.com/CV-2026-Project1>) is organized into four directories: *board+tiles*, *train*, *test*, and *evaluation*. All images were captured using a mobile phone under varying viewpoints, including changes in rotation, scale, and perspective. The *board+tiles* directory contains reference images of the empty board, as well as images in which all possible tile combinations are placed on the board. These images are intended to help you better understand the game configuration and can be used to extract useful visual information for your solution. The *train* and *test* directories share the same structure; however, the *test* data will be released after the submission deadline.

The *train* directory contains the following subdirectories:

- *boards* – contains 100 images and their corresponding annotation files, representing 5 games in which each player performs 10 moves (20 moves per game).

Each image corresponding to move i in game g is named $g_i.jpg$, where $g \in \{1, 2, 3, 4, 5\}$ and $i \in \{01, 02, 03, \dots, 20\}$. For each image, the corresponding annotation file has the same name with the extension `.txt`. All images are captured from different viewpoints.

In addition, for each game g , the file `g.table_score_mapping.txt` provides the correspondence between each board image and the associated table-score image. Each line in this file has the format:

`<game_image>    <table_score_image>`

where `<game_image>` is a file from the current game and `<table_score_image>` is the matching image from the *table-scores* collection. An example of such a mapping file is illustrated in Figure .

- *table-scores* – contains a collection of 200 score-table images extracted from multiple games. These images also exhibit variations in viewpoint and perspective, and serve as the candidate set from which the correct score table must be identified.

Data description

1_01.jpg	0100.jpg
1_02.jpg	0104.jpg
1_03.jpg	0057.jpg
1_04.jpg	0110.jpg
1_05.jpg	0167.jpg
1_06.jpg	0151.jpg
1_07.jpg	0129.jpg
1_08.jpg	0010.jpg
1_09.jpg	0085.jpg
1_10.jpg	0079.jpg
1_11.jpg	0044.jpg
1_12.jpg	0121.jpg
1_13.jpg	0177.jpg
1_14.jpg	0052.jpg
1_15.jpg	0165.jpg
1_16.jpg	0097.jpg
1_17.jpg	0191.jpg
1_18.jpg	0128.jpg
1_19.jpg	0145.jpg
1_20.jpg	0063.jpg

Figure 7: Example of a `g.table_score_mapping.txt` file. Each line specifies the correspondence between a board image (left) and the associated table-score image (right) from the table-scores collection.

In addition, for each game g , the file `g.table_score_mapping.txt` provides the correspondence between each board image and the associated table-score image. Each line in this file has the format:

`<game_image>    <table_score_image>`

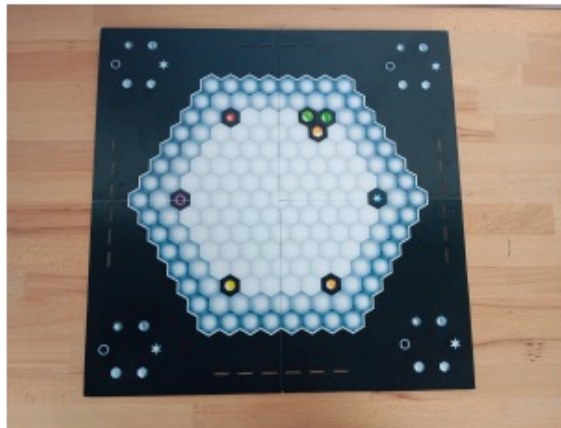
where `<game_image>` is a file from the current game and `<table_score_image>` is the matching image from the *table-scores* collection. An example of such a mapping file is illustrated in Figure 7.

- *table-scores* – contains a collection of 200 score-table images extracted from multiple games. These images also exhibit variations in viewpoint and perspective, and serve as the candidate set from which the correct score table must be identified.

Ground-truth annotations

The ground-truth annotation files provide, for each move, the following information:

- the positions of the two tile segments on the board, indexed from left to right and from top to bottom. Positions are encoded as integers in the range 1–85 (see Figure 3);
- the color of each segment of the placed tile, denoted by a letter: red (R), blue (B), green (G), yellow (Y), orange (O), and purple (P);
- the score obtained for that move. Since each tile contains exactly two colored segments, a player can score points for at most two colors in a single move. The score is encoded as pairs of the form kC , where k represents the number of points and C the corresponding color. For example, 1R or 2G 4B. The scores are listed following the fixed color order: Red, Green, Orange, Blue, Yellow, Purple, consistent with the representation used in the score tables.



```
1_01.txt - Notepad
File Edit Format View Help
4 G
10 O
1G
```

Figure 8: Example of a ground-truth annotation. The annotation specifies the positions of the two newly placed tile segments, their corresponding colors encoded as letters, and the score obtained for the move.

Test Data

- same as training data: 5 games, each game has 20 images
- testing data: 100 images (will be made available after the deadline for submitting the source code, on Monday 4th of May)

For each test image you should produce a similar txt file (like in the annotations) containing:

1. the positions of the tiles added to the board (numbers from 1 to 85, left to right, top to bottom)
2. the color of tiles
3. the score for each move
4. the corresponding image name of the table score from the collection of 200 table scores

Each correctly solved image worth 0.045 points:

- correct position: 0.015 points

- correct tile: 0.01 points

Total: 100 images x 0.025p = 2.5 points

- correct score after each turn: 0.005 points

Total: 100 images x 0.005p = 0.5 points

- corresponding image name of the table score: 0.015 points

Total: 100 images x 0.015p = 1.5 points

Ex officio: 0.5 points Correctformat + oral presentation + pdf (mandatory) : 0.5 point

Deadlines

Submit a *zip archive* containing your code (Python files or Jupyter notebook files), all auxiliary data that you are using (templates, models, etc.) and a pdf file describing your approach until Sunday, 3th of May, 11.59 PM using the following link <https://tinyurl.com/CV-2026-PROJECT1-SUBMISSIONS>. Please do not include in your zip archive any unuseful data (like training images, we already have them!!!). Notice that this is a hard deadline, no projects will be accepted after the deadline. Your code should include a README file (see the example in the materials for this project) containing the following information: (i) the libraries required to run the project including the full version of each library; (ii) indications of how to run the solution and where to look for the output file. Students who do not describe their approach (using a pdf file) will incur a penalty of 1 point.

On Monday 4th of May we will make available the test data. You will have to run your solution on the test images provided by us and upload your results in the same day as a zip archive using the following link <https://tinyurl.com/CV-2026-PROJECT1-RESULTS>.

Important note

IMPORTANT NOTE. After the deadline, you are not allowed to make any changes that improve your solution on the test data. This includes modifying code parameters, updating trained models, altering templates, or similar adjustments. Please ensure that you test your solution in a newly created environment following your README instructions to confirm that it runs correctly on our machines. We will execute your code to verify that your reported results match our outputs. You may only change non-critical settings, such as adjusting paths for test data or modifying try/except blocks if your code produces an error when processing a test image.

Materials for Project 1

tinyurl.com/CV-2026-Project1